

Chapter 1

Advanced Python Concepts and Best Practice



Introduction to Python Programming

- Python is a general purpose high level programming language.
- Python was developed by **Guido Van Rossum** in 1989 while working at National Research Institute at Netherlands.
- But officially Python was made available to public in 1991.
- The official Date of Birth for Python is : Feb 20th 1991.
- Python is recommended as first programming language for beginners.
- The name Python was selected from the TV Show "**The Complete Monty Python's Circus**", which was broadcasted in BBC from 1969 to 1974.

Where can we use Python Programming ?



- We can use everywhere. The most common important application areas are
- For developing Desktop Applications
- For developing web Applications
- For developing database Applications
- For Network Programming
- For developing games
- For Data Analysis Applications
- For Machine Learning
- For developing Artificial Intelligence Applications
- For IOT and so on...

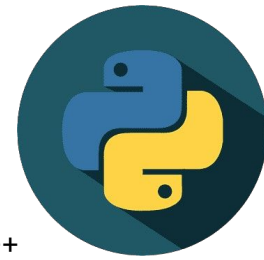
Simplicity in Python Programming

Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

C

```
#include <stdio.h>  
  
int main() {  
    printf("Hello, World!\n");  
    return 0;  
}
```



C++

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
    cout << "Hello, World!" << endl;  
    return 0;  
}
```

Python

```
print("Hello, World!")
```


Review of Python essentials and coding conventions

Identifiers

A name in Python program is called identifier.

It can be class name or function name or module name or variable name.

Eg : `a = 10`

Rules to define a identifier

1. The only allowed characters in Python are :

- alphabet symbols(either lower case or upper case)
- digits(0 to 9)
- underscore symbol(_)

By mistake if we are using any other symbol like \$ then we will get syntax error.

- cash = 10 (Correct)
- ca\$h =20 (Incorrect)

2. Identifier should not starts with digit

- 123total (Incorrect)
- total123 (Correct)

3. Identifiers are case sensitive. Of course Python language is case sensitive language.

```
total=10
```

```
TOTAL=999
```

```
print(total) #10
```

```
print(TOTAL) #999
```

Rules to define a identifier

1. Alphabet Symbols (Either Upper case OR Lower case)
2. If Identifier is start with Underscore (`_`) then it indicates it is private.
3. Identifier should not start with Digits.
4. Identifiers are case sensitive.
5. We cannot use reserved words as identifiers Eg: `def=10`
6. There is no length limit for Python identifiers. But not recommended to use too lengthy identifiers.
7. Dollor (\$) Symbol is not allowed in Python.

Rules to define a identifier

Q. Which of the following are valid Python identifiers?

- 1) 123total (Incorrect)
- 2) total123 (Correct)
- 3) java2share (Correct)
- 4) ca\$h (Incorrect)
- 5) _abc_abc_ (Correct)
- 6) def (Incorrect)
- 7) if (Incorrect)

Rules to define a identifier

Note :

- If identifier starts with `_` symbol then it indicates that it is private
- If identifier starts with `__`(two underscore symbols) indicating that strongly private identifier.
- If the identifier starts and ends with two underscore symbols then the identifier is language defined special name, which is also known as magic methods. Eg: `__add__`

Reserved Word(Keywords)

Python some words are reserved to represent some meaning or functionality. Such type of words are called Reserved words.

There are 33 reserved words available as of Python 3.10.

**True,False,None and, or ,not,is if,elif,else
while,for,break,continue,return,in,yield try,except,finally,raise,assert
import,from,as,class,def,pass,global,nonlocal,lambda,del,with**

Reserved Word(Keywords)

1. All Reserved words in Python contain only alphabet symbols.
2. Except the following 3 reserved words, all contain only lower case alphabet symbols.
 - True
 - False
 - None

Eg: a= true(Incorrect)

a=True (Correct)

```
>>> import keyword
```

```
>>> keyword.kwlist
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally',  
'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while',  
'with', 'yield']
```


Data Types

Data Type represent the type of data present inside a variable.

In Python we are not required to specify the type explicitly. Based on value provided,the type will be assigned automatically.

Hence Python is Dynamically Typed Language.

Data Types

Python contains the following inbuilt data types

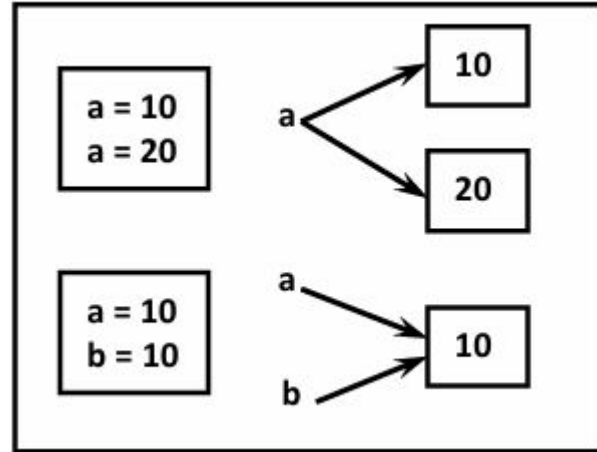
1. int 2. float 3.complex

4.bool 5.str 6.bytes

7.bytearray 8.range 9.list

10.tuple 11.set 12.frozenset

13.dict 14.None

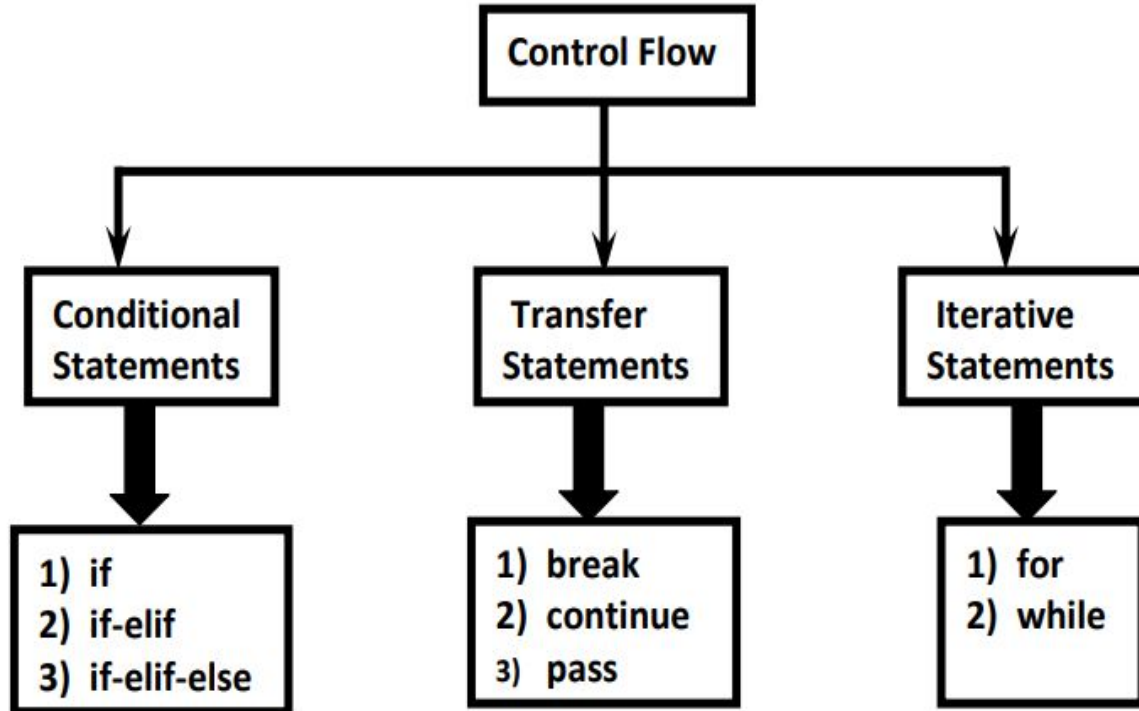


Some of the inbuilt function generally used :

- `type()` to check the type of variable
- `id()` to get address of object
- `print()` to print the value

Flow Control

Flow control describes the order in which statements will be executed at runtime.



Conditional Statements

1) if

if condition : statement

or

if condition :

statement-1

statement-2

statement-3

If condition is true then statements will be executed.

```
1) name=input("Enter Name:")
2) if name=="durga" :
3)     print("Hello Durga Good Morning")
4) print("How are you!!!")
5)
6) D:\Python_classes>py test.py
7) Enter Name:durga
8) Hello Durga Good Morning
9) How are you!!!
10)
11) D:\Python_classes>py test.py
12) Enter Name:Ravi
13) How are you!!!
```

Conditional Statements

2) if-else:

if condition :
 Action-1
else :
 Action-2

if condition is true then Action-1 will be executed otherwise Action-2 will be executed.

Eg:

```
1) name=input("Enter Name:")
2) if name=="durga" :
3)     print("Hello Durga Good Morning")
4) else:
5)     print("Hello Guest Good Moring")
6) print("How are you!!!")
7)
8) D:\Python_classes>py test.py
9) Enter Name:durga
10) Hello Durga Good Morning
11) How are you!!!
12)
13) D:\Python_classes>py test.py
14) Enter Name:Ravi
15) Hello Guest Good Moring
16) How are you!!!
```

Conditional Statements

3) if-elif-else:

Syntax:

if condition1:

 Action-1

elif condition2:

 Action-2

elif condition3:

 Action-3

elif condition4:

 Action-4

...

else:

 Default Action

Based condition the corresponding action will be executed.

```
1) brand=input("Enter Your Favourite Brand:")
2) if brand=="RC" :
3)     print("It is childrens brand")
4) elif brand=="KF":
5)     print("It is not that much kick")
6) elif brand=="FO":
7)     print("Buy one get Free One")
8) else :
9)     print("Other Brands are not recommended")
10)
11)
12) D:\Python_classes>py test.py
13) Enter Your Favourite Brand:RC
14) It is childrens brand
15)
16) D:\Python_classes>py test.py
17) Enter Your Favourite Brand:KF
18) It is not that much kick
19)
20) D:\Python_classes>py test.py
21) Enter Your Favourite Brand:KALYANI
```

1. else part is always optional

Hence the following are various possible syntaxes.

1. if

2. if - else

3. if-elif-else

4. if-elif

Iterative Statements (loop)

If we want to execute a group of statements multiple times then we should go for Iterative statements.

Python supports 2 types of iterative statements.

1. for loop
2. while loop

for loop

```
s="Dhiraj Pyakurel"  
for x in s :  
    print(x)
```

If we want to execute some action for every element present in some sequence(it may be string or collection)then we should go for for loop.

Syntax:

```
for x in sequence :  
    body
```

where sequence can be string or any collection.

Body will be executed for every element present in the sequence.

Eg 2: To print characters present in string index wise:

```
1) s=input("Enter some String: ")  
2) i=0  
3) for x in s :  
4)     print("The character present at ",i,"index is :",x)  
5)     i=i+1
```

Eg 3: To print Hello 10 times

```
1) for x in range(10) :  
2)     print("Hello")
```

for loop

Eg 4: To display numbers from 0 to 10

```
1) for x in range(11) :  
2) print(x)
```

Eg 5: To display odd numbers from 0 to 20

```
1) for x in range(21) :  
2) if (x%2!=0):  
3) print(x)
```

Eg 6: To display numbers from 10 to 1 in descending order

```
1) for x in range(10,0,-1) :  
2) print(x)
```

for loop

Eg 7: To print sum of numbers present inside list

```
1)      list=eval(input("Enter List:"))
2)      sum=0;
3)      for x in list:
4)          sum=sum+x;
5)      print("The Sum=",sum)
6)
7) D:\Python_classes>py test.py
8) Enter List:[10,20,30,40]
9) The Sum= 100
10)
11) D:\Python_classes>py test.py
12) Enter List:[45,67]
13) The Sum= 112
```

while loop

If we want to execute a group of statements iteratively until some condition false, then we should go for while loop.

Syntax:

```
while condition :  
    body
```

Eg: To print numbers from 1 to 10 by using while loop

```
1) x=1  
2) while x <=10:  
3) print(x)  
4) x=x+1
```

Eg: To display the sum of first n numbers

```
1) n=int(input("Enter number:"))  
2) sum=0  
3) i=1  
4) while i<=n:  
5)     sum=sum+i  
6)     i=i+1  
7) print("The sum of first",n,"numbers is :",sum)
```

while loop

Eg: write a program to prompt user to enter some name until entering Durga

```
1) name=""  
2) while name!="durga":  
3)     name=input("Enter Name:")  
4)     print("Thanks for confirmation")
```

Infinite Loops:

```
1) i=0;  
2) while True :  
3)     i=i+1;  
4)     print("Hello",i)
```

Nested loop

Sometimes we can take a loop inside another loop, which are also known as nested loops.

Eg:

```
1) for i in range(4):
2)     for j in range(4):
3)         print("i=",i, " j=",j)
4)
5) Output
6) D:\Python_classes>py test.py
7) i= 0  j= 0
8) i= 0  j= 1
9) i= 0  j= 2
10) i= 0  j= 3
11) i= 1  j= 0
12) i= 1  j= 1
13) i= 1  j= 2
14) i= 1  j= 3
15) i= 2  j= 0
16) i= 2  j= 1
17) i= 2  j= 2
18) i= 2  j= 3
19) i= 3  j= 0
20) i= 3  j= 1
21) i= 3  j= 2
22) i= 3  j= 3
```

Q. Write a program to display '*'s in Right angled triangled form

```
1) *
2) * *
3) * * *
4) * * * *
5) * * * * *
6) * * * * * *
7) * * * * * * *
8)
9) n = int(input("Enter number of rows:"))
10) for i in range(1,n+1):
11)     for j in range(1,i+1):
12)         print("*",end=" ")
13)     print()
```

Alternative way:

```
1) n = int(input("Enter number of rows:"))
2) for i in range(1,n+1):
3)     print("* " * i)
```

Transfer Statements (break)

We can use break statement inside loops to break loop execution based on some condition.

Eg:

Eg:

```
1) for i in range(10):
2)     if i==7:
3)         print("processing is enough..plz break")
4)         break
5)     print(i)
6)
7) D:\Python_classes>py test.py
8) 0
9) 1
10) 2
11) 3
12) 4
13) 5
14) 6
15) processing is enough..plz break
```

```
1) cart=[10,20,600,60,70]
2) for item in cart:
3)     if item>500:
4)         print("To place this order insurance must be required")
5)         break
6)     print(item)
7)
8) D:\Python_classes>py test.py
9) 10
10) 20
11) To place this order insurance must be required
```


Transfer Statement (Continue)

We can use continue statement to skip current iteration and continue next iteration.

Eg 1: To print odd numbers in the range 0 to 9

```
1) for i in range(10):
2)     if i%2==0:
3)         continue
4)     print(i)
5)
6) D:\Python_classes>py test.py
7) 1
8) 3
9) 5
10) 7
11) 9
```

Eg 2:

```
1) cart=[10,20,500,700,50,60]
2) for item in cart:
3)     if item>=500:
4)         print("We cannot process this item :",item)
5)         continue
6)     print(item)
7)
8) Output
9) D:\Python_classes>py test.py
10) 10
11) 20
12) We cannot process this item : 500
13) We cannot process this item : 700
14) 50
15) 60
```


Loop with else block

Inside loop execution, if break statement not executed, then only else part will be executed.

else means loop without break

Eg:

```
1) cart=[10,20,30,40,50]
2) for item in cart:
3)     if item>=500:
4)         print("We cannot process this order")
5)         break
6)     print(item)
7) else:
8)     print("Congrats ...all items processed successfully")
9)
```

10) Output

11) 10

12) 20

13) 30

14) 40

15) 50

16) Congrats ...all items processed successfully

```
1) cart=[10,20,600,30,40,50]
2) for item in cart:
3)     if item>=500:
4)         print("We cannot process this order")
5)         break
6)     print(item)
7) else:
8)     print("Congrats ...all items processed successfully")
```

Pass statement

pass is a keyword in Python.

In our programming syntactically if block is required which won't do anything then we can define that empty block with pass keyword.

pass

- |- It is an empty statement
- |- It is null statement
- |- It won't do anything

Eg:

if True:

SyntaxError: unexpected EOF while parsing

if True: pass

==>valid

def m1():

SyntaxError: unexpected EOF while parsing

```
def m1(): pass
```

use case of pass:

Sometimes in the parent class we have to declare a function with empty body and child class responsible to provide proper implementation. Such type of empty body we can define by using pass keyword. (It is something like abstract method in java)

Eg:

```
def m1(): pass
```

Eg:

```
1) for i in range(100):  
2)     if i%9==0:  
3)         print(i)  
4)     else:pass  
5)
```

del statement

del is a keyword in Python.

After using a variable, it is highly recommended to delete that variable if it is no longer required, so that the corresponding object is eligible for Garbage Collection.

We can delete variable by using del keyword.

Eg:

```
1) x=10  
2) print(x)  
3) del x
```

After deleting a variable we cannot access that variable otherwise we will get NameError.

Eg:

```
1) x=10  
2) del x  
3) print(x)
```

NameError: name 'x' is not defined.

Del statement

Note:

We can delete variables which are pointing to immutable objects. But we cannot delete the elements present inside immutable object.

Eg:

```
1) s="durga"
2) print(s)
3) del s==>valid
4) del s[0]==>TypeError: 'str' object doesn't support item deletion
```

Difference between del and None

Difference between del and None:

In the case del, the variable will be removed and we cannot access that variable(unbind operation)

```
1) s="durga"
2) del s
3) print(s)    ==>NameError: name 's' is not defined.
```

But in the case of None assignment the variable won't be removed but the corresponding object is eligible for Garbage Collection(re bind operation). Hence after assigning with None value,we can access that variable.

```
1) s="durga"
2) s=None
3) print(s)    # None
```

Functions

If a group of statements is repeatedly required then it is not recommended to write these statements everytime separately. We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting. This unit is nothing but function.

The main advantage of functions is code Reusability.

Note: In other languages functions are known as methods, procedures, subroutines etc

Python supports 2 types of functions

1. Built in Functions
2. User Defined Functions

Functions

1. Built in Functions:

The functions which are coming along with Python software automatically, are called built in functions or pre defined functions

Eg:

id()
type()
input()
eval()
etc..

2. User Defined Functions:

The functions which are developed by programmer explicitly according to business requirements, are called user defined functions.

Syntax to create user defined functions:

```
def function_name(parameters) :  
    """ doc string """  
    ----  
    -----  
    return value
```

Note: While creating functions we can use 2 keywords

1. def (mandatory)
2. return (optional)

Eg 1: Write a function to print Hello

test.py:

```
1) def wish():  
2)     print("Hello Good Morning")  
3) wish()  
4) wish()  
5) wish()
```


Functions

Parameters

Parameters are inputs to the function. If a function contains parameters, then at the time of calling, compulsory we should provide values otherwise, otherwise we will get error.

Eg: Write a function to take name of the student as input and print wish message by name.

```
1. def wish(name):
2.     print("Hello",name, " Good Morning")
3. wish("Durga")
4. wish("Ravi")
5.
6.
7. D:\Python_classes>py test.py
8. Hello Durga Good Morning
9. Hello Ravi Good Morning
```

Eg: Write a function to take number as input and print its square value

```
1. def squarelt(number):
2.     print("The Square of",number,"is", number*number)
3. squarelt(4)
4. squarelt(5)
5.
6. D:\Python_classes>py test.py
7. The Square of 4 is 16
8. The Square of 5 is 25
```

Return Statement:

Function can take input values as parameters and executes business logic, and returns output to the caller with return statement.

Q. Write a function to accept 2 numbers as input and return sum.

```
1. def add(x,y):
2.     return x+y
3. result=add(10,20)
4. print("The sum is",result)
5. print("The sum is",add(100,200))
6.
7.
8. D:\Python_classes>py test.py
9. The sum is 30
10. The sum is 300
```

If we are not writing return statement then default return value is None

Eg:

```
1. def f1():
2.     print("Hello")
3. f1()
4. print(f1())
5.
6. Output
7. Hello
8. Hello
9. None
```


Functions

Q. Write a function to check whether the given number is even or odd?

```
1. def even_odd(num):
2.     if num%2==0:
3.         print(num,"is Even Number")
4.     else:
5.         print(num,"is Odd Number")
6. even_odd(10)
7. even_odd(15)
8.
9. Output
10. D:\Python_classes>py test.py
11. 10 is Even Number
12. 15 is Odd Number
```

Q. Write a function to find factorial of given number?

```
1) def fact(num):
2)     result=1
3)     while num>=1:
4)         result=result*num
5)         num=num-1
6)     return result
7) for i in range(1,5):
8)     print("The Factorial of",i,"is :",fact(i))
```

Functions

Returning multiple values from a function:

In other languages like C,C++ and Java, function can return atmost one value. But in Python, a function can return any number of values.

Eg 1:

```
1) def sum_sub(a,b):
2)     sum=a+b
3)     sub=a-b
4)     return sum,sub
5) x,y=sum_sub(100,50)
6) print("The Sum is :",x)
7) print("The Subtraction is :",y)
8)
9) Output
10) The Sum is : 150
11) The Subtraction is : 50
```

Eg 2:

```
1) def calc(a,b):
2)     sum=a+b
3)     sub=a-b
4)     mul=a*b
5)     div=a/b
6)     return sum,sub,mul,div
7) t=calc(100,50)
8) print("The Results are")
9) for i in t:
10)     print(i)
```

Functions

Types of arguments

```
def f1(a,b):  
    -----  
    -----  
    -----  
f1(10,20)
```

a,b are formal arguments where as 10,20 are actual arguments

There are 4 types of arguments allowed in Python.

1. positional arguments
2. keyword arguments
3. default arguments
4. Variable length arguments

1. positional arguments:

These are the arguments passed to function in correct positional order.

```
def sub(a,b):  
    print(a-b)  
  
sub(100,200)  
sub(200,100)
```

The number of arguments and position of arguments must be matched. If we change the order then result may be changed.

If we change the number of arguments then we will get error.

Functions

2. keyword arguments:

We can pass argument values by keyword i.e by parameter name.

Eg:

```
1. def wish(name,msg):  
2.     print("Hello",name,msg)  
3. wish(name="Durga",msg="Good Morning")  
4. wish(msg="Good Morning",name="Durga")  
5.  
6. Output  
7. Hello Durga Good Morning  
8. Hello Durga Good Morning
```

Here the order of arguments is not important but number of arguments must be matched.

Note:

We can use both positional and keyword arguments simultaneously. But first we have to take positional arguments and then keyword arguments, otherwise we will get syntaxerror.

```
def wish(name,msg):  
    print("Hello",name,msg)  
wish("Durga","GoodMorning")    ==>valid  
wish("Durga",msg="GoodMorning") ==>valid  
wish(name="Durga","GoodMorning") ==>invalid  
SyntaxError: positional argument follows keyword argument
```

Functions

3. Default Arguments:

Sometimes we can provide default values for our positional arguments.

Eg:

```
1) def wish(name="Guest"):
2)     print("Hello",name,"Good Morning")
3)
4) wish("Durga")
5) wish()
6)
7) Output
8) Hello Durga Good Morning
9) Hello Guest Good Morning
```

If we are not passing any name then only default value will be considered.

***Note:

After default arguments we should not take non default arguments

```
def wish(name="Guest",msg="Good Morning"): ==>Valid
def wish(name,msg="Good Morning"): ==>Valid
def wish(name="Guest",msg): ==>Invalid
```

SyntaxError: non-default argument follows default argument

Functions

4. Variable length arguments:

Sometimes we can pass variable number of arguments to our function, such type of arguments are called variable length arguments.

We can declare a variable length argument with * symbol as follows

```
def f1(*n):
```

We can call this function by passing any number of arguments including zero number. Internally all these values represented in the form of tuple.

Eg:

```
1) def sum(*n):  
2)     total=0  
3)     for n1 in n:  
4)         total=total+n1  
5)     print("The Sum=",total)  
6)  
7) sum()  
8) sum(10)  
9) sum(10,20)  
10) sum(10,20,30,40)  
11)  
12) Output  
13) The Sum= 0  
14) The Sum= 10  
15) The Sum= 30  
16) The Sum= 100
```

Functions

Note:

We can mix variable length arguments with positional arguments.

```
1) def f1(n1,*s):
2)     print(n1)
3)     for s1 in s:
4)         print(s1)
5)
6) f1(10)
7) f1(10,20,30,40)
8) f1(10,"A",30,"B")
9)
10) Output
11) 10
12) 10
13) 20
14) 30
15) 40
16) 10
17) A
18) 30
19) B
```

Note: After variable length argument, if we are taking any other arguments then we should provide values as keyword arguments.

Eg:

```
1) def f1(*s,n1):
2)     for s1 in s:
3)         print(s1)
4)         print(n1)
5)
6) f1("A", "B", n1=10)
7) Output
8) A
9) B
10) 10
```

f1("A","B",10) ==>Invalid

TypeError: f1() missing 1 required keyword-only argument: 'n1'

Note: We can declare key word variable length arguments also.
For this we have to use **.

Functions

Note: We can declare key word variable length arguments also. For this we have to use **.

```
def f1(**n):
```

We can call this function by passing any number of keyword arguments. Internally these keyword arguments will be stored inside a dictionary.

Eg:

```
1) def display(**kwargs):
2)     for k,v in kwargs.items():
3)         print(k,"=",v)
4) display(n1=10,n2=20,n3=30)
5) display(rno=100,name="Durga",marks=70,subject="Java")
6)
7) Output
8) n1 = 10
9) n2 = 20
10) n3 = 30
11) rno = 100
12) name = Durga
13) marks = 70
14) subject = Java
```


Functions

Case Study:

```
def f(arg1,arg2,arg3=4,arg4=8):  
    print(arg1,arg2,arg3,arg4)
```

1. `f(3,2) ==> 3 2 4 8`

2. `f(10,20,30,40) ==> 10 20 30 40`

3. `f(25,50,arg4=100) ==> 25 50 4 100`

4. `f(arg4=2,arg1=3,arg2=4) ==> 3 4 4 2`

5. `f() ==> Invalid`
`TypeError: f() missing 2 required positional arguments: 'arg1' and 'arg2'`

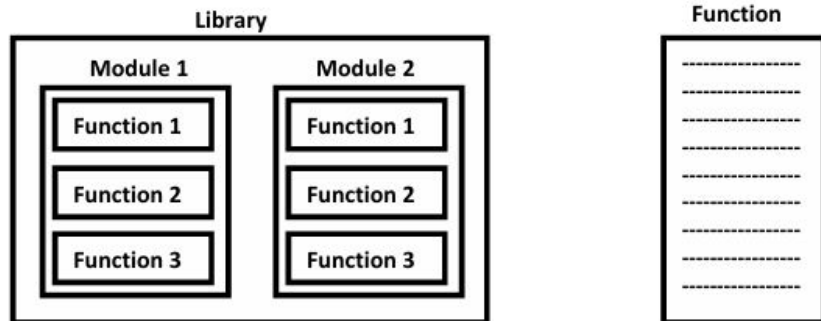
6. `f(arg3=10,arg4=20,30,40) ==> Invalid`
`SyntaxError: positional argument follows keyword argument`
[After keyword arguments we should not take positional arguments]

7. `f(4,5,arg2=6) ==> Invalid`
`TypeError: f() got multiple values for argument 'arg2'`

8. `f(4,5,arg3=5,arg5=6) ==> Invalid`
`TypeError: f() got an unexpected keyword argument 'arg5'`

Note: Function vs Module vs Library:

1. A group of lines with some name is called a function
2. A group of functions saved to a file, is called Module
3. A group of Modules is nothing but Library



Functions

global keyword:

We can use global keyword for the following 2 purposes:

1. To declare global variable inside function
2. To make global variable available to the function so that we can perform required modifications

Eg 1:

```
1) a=10
2) def f1():
3)     a=777
4)     print(a)
5)
6) def f2():
7)     print(a)
8)
9) f1()
10) f2()
11)
12) Output
13) 777
14) 10
```

Eg 2:

```
1) a=10
2) def f1():
3)     global a
4)     a=777
5)     print(a)
6)
7) def f2():
8)     print(a)
9)
10) f1()
11) f2()
12)
13) Output
14) 777
15) 777
```

Eg 3:

```
1) def f1():
2)     a=10
3)     print(a)
4)
5) def f2():
6)     print(a)
7)
8) f1()
9) f2()
10)
11) NameError: name 'a' is not defined
```

Eg 4:

```
1) def f1():
2)     global a
3)     a=10
4)     print(a)
5)
6) def f2():
7)     print(a)
8)
9) f1()
10) f2()
11)
12) Output
13) 10
14) 10
```

Functions

Note: If global variable and local variable having the same name then we can access global variable inside a function as follows

```
1) a=10  #global variable
2) def f1():
3)     a=777 #local variable
4)     print(a)
5)     print(globals()['a'])
6) f1()
7)
8)
9) Output
10) 777
11) 10
```

Functions

Recursive Functions

A function that calls itself is known as Recursive Function.

Eg:

```
factorial(3)=3*factorial(2)
            =3*2*factorial(1)
            =3*2*1*factorial(0)
            =3*2*1*1
            =6
factorial(n)= n*factorial(n-1)
```

The main advantages of recursive functions are:

1. We can reduce length of the code and improves readability
2. We can solve complex problems very easily.

Q. Write a Python Function to find factorial of given number with recursion.

Eg:

```
1) def factorial(n):
2)   if n==0:
3)     result=1
4)   else:
5)     result=n*factorial(n-1)
6)   return result
7) print("Factorial of 4 is :",factorial(4))
8) print("Factorial of 5 is :",factorial(5))
```

Functions

Anonymous Functions:

Sometimes we can declare a function without any name, such type of nameless functions are called anonymous functions or lambda functions.

The main purpose of anonymous function is just for instant use (i.e. for one time usage)

Normal Function:

We can define by using def keyword.

```
def squarelt(n):  
    return n*n
```

lambda Function:

We can define by using lambda keyword

```
lambda n:n*n
```

Syntax of lambda Function:

```
lambda argument_list : expression
```

Note: By using Lambda Functions we can write very concise code so that readability of the program will be improved.

Functions

Q. Write a program to create a lambda function to find square of given number?

- 1) `s=lambda n:n*n`
- 2) `print("The Square of 4 is :",s(4))`
- 3) `print("The Square of 5 is :",s(5))`
- 4)
- 5) Output
- 6) The Square of 4 is : 16
- 7) The Square of 5 is : 25

Q. Lambda function to find sum of 2 given numbers

- 1) `s=lambda a,b:a+b`
- 2) `print("The Sum of 10,20 is:",s(10,20))`
- 3) `print("The Sum of 100,200 is:",s(100,200))`

Q. Lambda Function to find biggest of given values.

- 1) `s=lambda a,b:a if a>b else b`
- 2) `print("The Biggest of 10,20 is:",s(10,20))`
- 3) `print("The Biggest of 100,200 is:",s(100,200))`
- 4)
- 5) Output
- 6) The Biggest of 10,20 is: 20
- 7) The Biggest of 100,200 is: 200

Note:

Lambda Function internally returns expression value and we are not required to write return statement explicitly.

Note: Sometimes we can pass function as argument to another function. In such cases lambda functions are best choice.

We can use lambda functions very commonly with `filter()`, `map()` and `reduce()` functions, b'z these functions expect function as argument.

Functions

filter() function:

We can use filter() function to filter values from the given sequence based on some condition.

`filter(function,sequence)`

where function argument is responsible to perform conditional check
sequence can be list or tuple or string.

Q. Program to filter only even numbers from the list by using filter() function?

without lambda Function:

```
1) def isEven(x):  
2)     if x%2==0:  
3)         return True  
4)     else:  
5)         return False  
6) l=[0,5,10,15,20,25,30]  
7) l1=list(filter(isEven,l))  
8) print(l1) #[0,10,20,30]
```

with lambda Function:

```
1) l=[0,5,10,15,20,25,30]  
2) l1=list(filter(lambda x:x%2==0,l))  
3) print(l1) #[0,10,20,30]  
4) l2=list(filter(lambda x:x%2!=0,l))  
5) print(l2) #[5,15,25]
```

Functions

map() function:

For every element present in the given sequence, apply some functionality and generate new element with the required modification. For this requirement we should go for map() function.

Eg: For every element present in the list perform double and generate new list of doubles.

Syntax:

map(function, sequence)

The function can be applied on each element of sequence and generates new sequence.

Eg: Without lambda

```
1) l=[1,2,3,4,5]
2) def doublelt(x):
3)     return 2*x
4) l1=list(map(doublelt,l))
5) print(l1) #[2, 4, 6, 8, 10]
```

with lambda

```
1) l=[1,2,3,4,5]
2) l1=list(map(lambda x:2*x,l))
3) print(l1)  #[2, 4, 6, 8, 10]
```


Functions

Eg 2: To find square of given numbers

```
1. l=[1,2,3,4,5]
2. l1=list(map(lambda x:x*x,l))
3. print(l1)    #[1, 4, 9, 16, 25]
```

We can apply map() function on multiple lists also. But make sure all list should have same length.

Syntax: map(lambda x,y:x*y,l1,l2))
x is from l1 and y is from l2

Eg:

```
1. l1=[1,2,3,4]
2. l2=[2,3,4,5]
3. l3=list(map(lambda x,y:x*y,l1,l2))
4. print(l3)    #[2, 6, 12, 20]
```

Functions

reduce() function:

reduce() function reduces sequence of elements into a single element by applying the specified function.

reduce(function,sequence)

reduce() function present in functools module and hence we should write import statement.

Eg:

```
1) from functools import *
2) l=[10,20,30,40,50]
3) result=reduce(lambda x,y:x+y,l)
4) print(result) # 150
```

Eg:

```
1) result=reduce(lambda x,y:x*y,l)
2) print(result) #12000000
```

Eg:

```
1) from functools import *
2) result=reduce(lambda x,y:x+y,range(1,101))
3) print(result) #5050
```

Note:

- In Python every thing is treated as object.
- Even functions also internally treated as objects only.

Eg:

```
1) def f1():
2)     print("Hello")
3) print(f1)
4) print(id(f1))
```

Output

```
<function f1 at 0x00419618>
4298264
```

Functions

Function Aliasing:

For the existing function we can give another name, which is nothing but function aliasing.

Eg:

```
1) def wish(name):
2)     print("Good Morning:",name)
3)
4) greeting=wish
5) print(id(wish))
6) print(id(greeting))
7)
8) greeting('Durga')
9) wish('Durga')
```

Output

4429336

4429336

Good Morning: Durga

Good Morning: Durga

Note: In the above example only one function is available but we can call that function by using either wish name or greeting name.

If we delete one name still we can access that function by using alias name

Eg:

```
1) def wish(name):
2)     print("Good Morning:",name)
3)
4) greeting=wish
5)
6) greeting('Durga')
7) wish('Durga')
8)
9) del wish
10) #wish('Durga') ==>NameError: name 'wish' is not defined
11) greeting('Pavan')
```

Functions

Nested Functions:

We can declare a function inside another function, such type of functions are called Nested functions.

Eg:

```
1) def outer():
2)     print("outer function started")
3)     def inner():
4)         print("inner function execution")
5)     print("outer function calling inner function")
6)     inner()
7) outer()
8) #inner()    ==>NameError: name 'inner' is not defined
```

Output

outer function started
outer function calling inner function
inner function execution

In the above example inner() function is local to outer() function and hence it is not possible to call directly from outside of outer() function.

Note: A function can return another function.

Eg:

```
1) def outer():
2)     print("outer function started")
3)     def inner():
4)         print("inner function execution")
5)     print("outer function returning inner function")
6)     return inner
7) f1=outer()
8) f1()
9) f1()
10) f1()
```

Functions

Q. What is the difference between the following lines?

```
f1 = outer  
f1 = outer()
```

- In the first case for the `outer()` function we are providing another name `f1` (function aliasing).
- But in the second case we are calling `outer()` function, which returns inner function. For that inner function() we are providing another name `f1`.

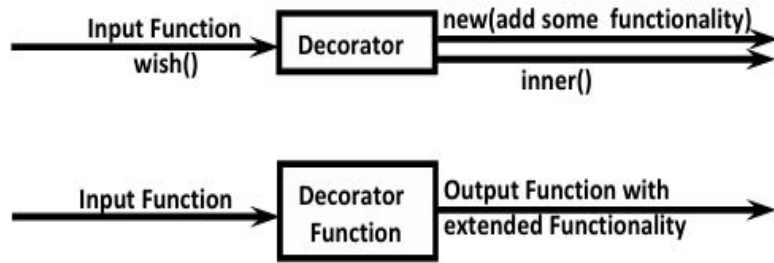
Note: We can pass function as argument to another function

Eg: `filter(function, sequence)`
`map(function, sequence)`
`reduce(function, sequence)`

Functions

Function Decorators:

Decorator is a function which can take a function as argument and extend its functionality and returns modified function with extended functionality.



The main objective of decorator functions is we can extend the functionality of existing functions without modifies that function.

```
1) def wish(name):
2)     print("Hello",name,"Good Morning")
```

This function can always print same output for any name

Hello Durga Good Morning
Hello Ravi Good Morning
Hello Sunny Good Morning

But we want to modify this function to provide different message if name is Sunny. We can do this without touching wish() function by using decorator.

Eg:

```
1) def decor(func):
2)     def inner(name):
3)         if name=="Sunny":
4)             print("Hello Sunny Bad Morning")
5)         else:
6)             func(name)
7)     return inner
8)
9) @decor
10) def wish(name):
11)     print("Hello",name,"Good Morning")
12)
13) wish("Durga")
14) wish("Ravi")
15) wish("Sunny")
16)
```

```
17) Output
18) Hello Durga Good Morning
19) Hello Ravi Good Morning
20) Hello Sunny Bad Morning
```

In the above program whenever we call wish() function automatically decor function will be executed.

Functions

How to call same function with decorator and without decorator:

We should not use @decor

```
1) def decor(func):
2)     def inner(name):
3)         if name=="Sunny":
4)             print("Hello Sunny Bad Morning")
5)         else:
6)             func(name)
7)     return inner
8)
9)
10) def wish(name):
11)     print("Hello",name,"Good Morning")
12)
13) decorfunction=decor(wish)
14)
15) wish("Durga") #decorator wont be executed
16) wish("Sunny") #decorator wont be executed
17)
18) decorfunction("Durga")#decorator will be executed
19) decorfunction("Sunny")#decorator will be executed
20)
21) Output
22) Hello Durga Good Morning
23) Hello Sunny Good Morning
24) Hello Durga Good Morning
25) Hello Sunny Bad Morning
```

Eg 2:

```
1) def smart_division(func):
2)     def inner(a,b):
3)         print("We are dividing",a,"with",b)
4)         if b==0:
5)             print("OOPS...cannot divide")
6)             return
7)         else:
8)             return func(a,b)
9)     return inner
10)
11) @smart_division
12) def division(a,b):
13)     return a/b
14)
15) print(division(20,2))
16) print(division(20,0))
17)
18) without decorator we will get Error.In this case output is:
19)
20) 10.0
21) Traceback (most recent call last):
22)   File "test.py", line 16, in <module>
23)     print(division(20,0))
24)   File "test.py", line 13, in division
25)     return a/b
26) ZeroDivisionError: division by zero
```

with decorator we won't get any error. In this case output is:

```
We are dividing 20 with 2
10.0
We are dividing 20 with 0
OOPS...cannot divide
None
```

Functions

Decorator Chaining

We can define multiple decorators for the same function and all these decorators will form Decorator Chaining.

Eg:

```
@decor1  
@decor  
def num():
```

For num() function we are applying 2 decorator functions. First inner decorator will work and then outer decorator.

Eg:

```
1) def decor1(func):  
2)     def inner():  
3)         x=func()  
4)         return x*x  
5)     return inner  
6)  
7) def decor(func):  
8)     def inner():  
9)         x=func()  
10)        return 2*x  
11)    return inner  
12)  
13) @decor1  
14) @decor  
15) def num():  
16)     return 10  
17)  
18) print(num())
```


Functions

Demo Program for decorator Chaining:

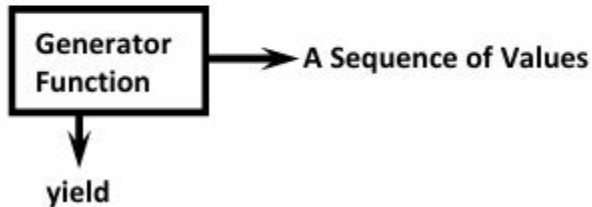
```
1) def decor(func):
2)     def inner(name):
3)         print("First Decor(decor) Function Execution")
4)         func(name)
5)     return inner
6)
7) def decor1(func):
8)     def inner(name):
9)         print("Second Decor(decor1) Execution")
10)        func(name)
11)    return inner
12)
13) @decor1
14) @decor
15) def wish(name):
16)     print("Hello",name,"Good Morning")
17)
18) wish("Durga")
```

Functions

Generators

Generator is a function which is responsible to generate a sequence of values.

We can write generator functions just like ordinary functions, but it uses yield keyword to return values.



Eg 1:

```
1) def mygen():
2)     yield 'A'
3)     yield 'B'
4)     yield 'C'
5)
6) g=mygen()
7) print(type(g))
8)
9) print(next(g))
10) print(next(g))
11) print(next(g))
12) print(next(g))
13)
14) Output
15) <class 'generator'>
16) A
17) B
18) C
19) Traceback (most recent call last):
20) File "test.py", line 12, in <module>
21)     print(next(g))
22) StopIteration
```

Functions

Eg 2:

```
1) def countdown(num):
2)     print("Start Countdown")
3)     while(num>0):
4)         yield num
5)         num=num-1
6)
7) values=countdown(5)
8) for x in values:
9)     print(x)
10)
11) Output
12) Start Countdown
13) 5
14) 4
15) 3
16) 2
17) 1
```

Eg 3: To generate first n numbers:

```
1) def firstn(num):
2)     n=1
3)     while n<=num:
4)         yield n
5)         n=n+1
6)
7) values=firstn(5)
8) for x in values:
9)     print(x)
10)
11) Output
12) 1
13) 2
14) 3
15) 4
16) 5
```

We can convert generator into list as follows:

```
values=firstn(10)
l1=list(values)
print(l1)  #[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Eg 4: To generate Fibonacci Numbers...

The next is the sum of previous 2 numbers

Eg: 0,1,1,2,3,5,8,13,21,...

```
1) def fib():
2)     a,b=0,1
3)     while True:
4)         yield a
5)         a,b=b,a+b
6) for f in fib():
7)     if f>100:
8)         break
9)     print(f)
```

Advantages of Generator Functions:

1. when compared with class level iterators, generators are very easy to use
2. Improves memory utilization and performance.
3. Generators are best suitable for reading data from large number of large files
4. Generators work great for web scraping and crawling.

Functions

Generators vs Normal Collections wrt performance:

```
1) import random
2) import time
3)
4) names = ['Sunny', 'Bunny', 'Chinny', 'Vinny']
5) subjects = ['Python', 'Java', 'Blockchain']
6)
7) def people_list(num_people):
8)     results = []
9)     for i in range(num_people):
10)         person = {
11)             'id': i,
12)             'name': random.choice(names),
13)             'subject': random.choice(subjects)
14)         }
15)         results.append(person)
16)     return results
17)
18) def people_generator(num_people):
19)     for i in range(num_people):
20)         person = {
21)             'id': i,
22)             'name': random.choice(names),
23)             'major': random.choice(subjects)
24)         }
25)         yield person
26)
27) """t1 = time.clock()
28) people = people_list(10000000)
29) t2 = time.clock()"""
30)
31) t1 = time.clock()
32) people = people_generator(10000000)
33) t2 = time.clock()
34)
35) print('Took {}'.format(t2-t1))
```

Note: In the above program observe the difference wrt execution time by using list and generators

Generators vs Normal Collections wrt Memory Utilization:

Normal Collection:

```
l=[x*x for x in range(10000000000000000)]
print(l[0])
```

We will get MemoryError in this case because all these values are required to store in the memory.

Generators:

```
g=(x*x for x in range(10000000000000000))
print(next(g))
```

Output: 0

We won't get any MemoryError because the values won't be stored at the beginning

Modules

A group of functions, variables and classes saved to a file, which is nothing but module.

Every Python file (.py) acts as a module.

Eg: durgamath.py

```
1) x=888
2)
3) def add(a,b):
4)     print("The Sum:",a+b)
5)
6) def product(a,b):
7)     print("The Product:",a*b)
```

durgamath module contains one variable and 2 functions.

Modules

If we want to use members of module in our program then we should import that module.

```
import modulename
```

We can access members by using module name.

```
modulename.variable
```

```
modulename.function()
```

test.py:

```
1) import durgamath
2) print(durgamath.x)
3) durgamath.add(10,20)
4) durgamath.product(10,20)
5)
6) Output
7) 888
8) The Sum: 30
9) The Product: 200
```

Note:

whenever we are using a module in our program, for that module compiled file will be generated and stored in the hard disk permanently.

Modules

Renaming a module at the time of import (module aliasing):

Eg:

```
import durgamath as m
```

here durgamath is original module name and m is alias name.

We can access members by using alias name m

test.py:

```
1) import durgamath as m
2) print(m.x)
3) m.add(10,20)
4) m.product(10,20)
```

Modules

from ... import:

We can import particular members of module by using from ... import .

The main advantage of this is we can access members directly without using module name.

Eg:

```
from durgamath import x,add
```

```
print(x)
```

```
add(10,20)
```

```
product(10,20)==> NameError: name 'product' is not defined
```

We can import all members of a module as follows

```
from durgamath import *
```

test.py:

```
1) from durgamath import *
```

```
2) print(x)
```

```
3) add(10,20)
```

```
4) product(10,20)
```


Modules

Various possibilities of import:

`import modulename`

`import module1,module2,module3`

`import module1 as m`

`import module1 as m1,module2 as m2,module3`

`from module import member`

`from module import member1,member2,member3`

`from module import member1 as x`

`from module import *`

Modules

member aliasing:

```
from durgamath import x as y, add as sum  
print(y)  
sum(10, 20)
```

Once we defined as alias name, we should use alias name only and we should not use original name

Eg:

```
from durgamath import x as y  
print(x) ==> NameError: name 'x' is not defined
```

Modules

Reloading a Module:

By default module will be loaded only once eventhough we are importing multiple multiple times.

Demo Program for module reloading:

```
1) import time
2) from imp import reload
3) import module1
4) time.sleep(30)
5) reload(module1)
6) time.sleep(30)
7) reload(module1)
8) print("This is test file")
```

Note: In the above program, everytime updated version of module1 will be available to our program

Modules

Finding members of module by using dir() function:

Python provides inbuilt function `dir()` to list out all members of current module or a specified module.

`dir()` ==> To list out all members of current module

`dir(moduleName)` ==> To list out all members of specified module

Eg 1: test.py

```
1) x=10
2) y=20
3) def f1():
4)     print("Hello")
5) print(dir()) # To print all members of current module
6)
7) Output
8) ['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'f1', 'x', 'y']
```

Modules

Eg 2: To display members of particular module:

durgamath.py:

```
1) x=888
2)
3) def add(a,b):
4)     print("The Sum:",a+b)
5)
6) def product(a,b):
7)     print("The Product:",a*b)
```

test.py:

```
1) import durgamath
2) print(dir(durgamath))
3)
4) Output
5) ['__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',
6)  '__package__', '__spec__', 'add', 'product', 'x']
```

Note: For every module at the time of execution Python interpreter will add some special properties automatically for internal use.

Modules

The Special variable `__name__`:

For every Python program , a special variable `__name__` will be added internally. This variable stores information regarding whether the program is executed as an individual program or as a module.

If the program executed as an individual program then the value of this variable is `__main__`

If the program executed as a module from some other program then the value of this variable is the name of module where it is defined.

Hence by using this `__name__` variable we can identify whether the program executed directly or as a module.

Modules

Demo program:

module1.py:

```
1) def f1():  
2)     if __name__ == '__main__':  
3)         print("The code executed as a program")  
4)     else:  
5)         print("The code executed as a module from some other program")  
6) f1()
```

test.py:

```
1) import module1  
2) module1.f1()  
3)  
4) D:\Python_classes>py module1.py  
5) The code executed as a program  
6)  
7) D:\Python_classes>py test.py  
8) The code executed as a module from some other program
```

Modules

Working with random module:

This module defines several functions to generate random numbers.

We can use these functions while developing games, in cryptography and to generate random numbers on fly for authentication.

1. random() function:

This function always generate some float value between 0 and 1 (not inclusive)

$$0 < x < 1$$

Eg:

```
1) from random import *
2) for i in range(10):
3)     print(random())
4)
5) Output
6) 0.4572685609302056
7) 0.6584325233197768
8) 0.15444034016553587
9) 0.18351427005232201
10) 0.1330257265904884
11) 0.9291139798071045
12) 0.6586741197891783
13) 0.8901649834019002
14) 0.25540891083913053
15) 0.7290504335962871
```


Modules

2. randint() function:

To generate random integer between two given numbers(inclusive)

Eg:

```
1) from random import *
2) for i in range(10):
3)     print(randint(1,100)) # generate random int value between 1 and 100(inclusive)
4)
5) Output
6) 51
7) 44
8) 39
9) 70
10) 49
11) 74
12) 52
13) 10
14) 40
15) 8
```

Modules

3. uniform():

It returns random float values between 2 given numbers(not inclusive)

Eg:

```
1) from random import *
2) for i in range(10):
3)     print(uniform(1,10))
4)
5) Output
6) 9.787695398230332
7) 6.81102218793548
8) 8.068672144377329
9) 8.567976357239834
10) 6.363511674803802
11) 2.176137584071641
12) 4.822867939432386
13) 6.0801725149678445
14) 7.508457735544763
15) 1.9982221862917555
```

random() ==> in between 0 and 1 (not inclusive)

randint(x,y) ==> in between x and y (inclusive)

uniform(x,y) ==> in between x and y (not inclusive)

Modules

4. randrange([start].stop.[step])

returns a random number from range

start<= x < stop

start argument is optional and default value is 0

step argument is optional and default value is 1

randrange(10)-->generates a number from 0 to 9

randrange(1,11)-->generates a number from 1 to 10

randrange(1,11,2)-->generates a number from 1,3,5,7,9

Eg 1:

```
1) from random import *
2) for i in range(10):
3)     print(randrange(10))
4)
5) Output
6) 9
7) 4
8) 0
9) 2
10) 9
11) 4
12) 8
13) 9
14) 5
15) 9
```

Modules

5. choice() function:

It wont return random number.

It will return a random object from the given list or tuple.

Eg:

```
1) from random import *  
2) list=["Sunny","Bunny","Chinny","Vinny","pinny"]  
3) for i in range(10):  
4)     print(choice(list))
```

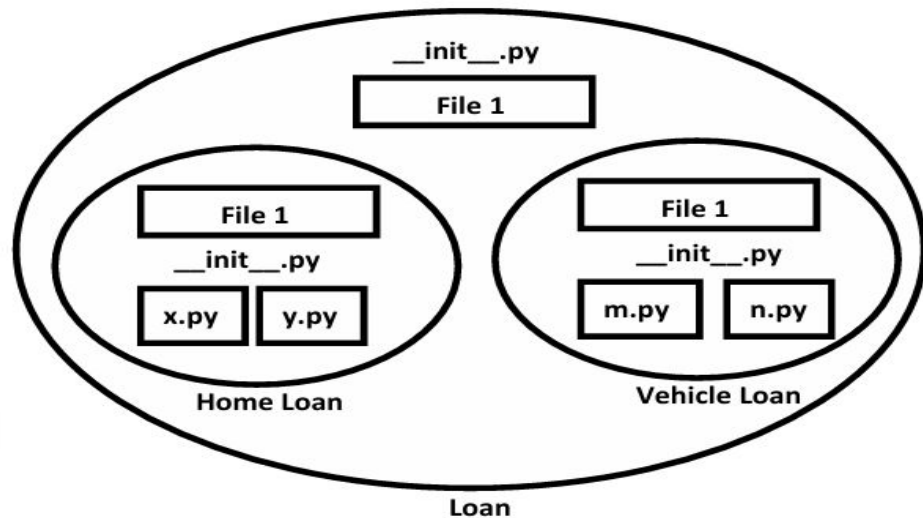
Packages

It is an encapsulation mechanism to group related modules into a single unit.

package is nothing but folder or directory which represents collection of Python modules.

Any folder or directory contains `__init__.py` file, is considered as a Python package. This file can be empty.

A package can contain sub packages also.



Packages

The main advantages of package statement are

1. We can resolve naming conflicts
2. We can identify our components uniquely
3. It improves modularity of the application

Packages

module1.py:

```
def f1():  
    print("Hello this is from module1 present in pack1")
```

test.py (version-1):

```
import pack1.module1  
pack1.module1.f1()
```

test.py (version-2):

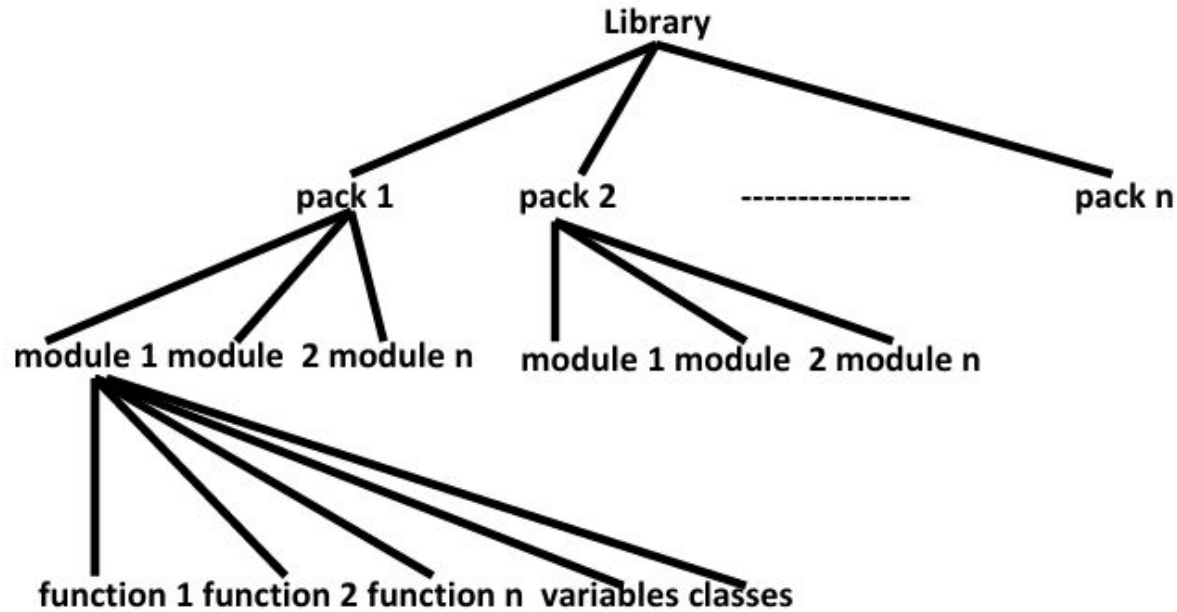
```
from pack1.module1 import f1  
f1()
```

Packages

test.py:

1. `from com.module1 import f1`
2. `from com.durgasoft.module2 import f2`
3. `f1()`
4. `f2()`
- 5.
6. Output
7. `D:\Python_classes>py test.py`
8. Hello this is from module1 present in com
9. Hello this is from module2 present in com.durgasoft

Packages



Exception Handling

In any programming language there are 2 types of errors are possible.

1. Syntax Errors
2. Runtime Errors

1. Syntax Errors:

The errors which occurs because of invalid syntax are called syntax errors.

Eg 1:

```
x=10
if x==10
    print("Hello")
```

SyntaxError: invalid syntax

Note:

Programmer is responsible to correct these syntax errors. Once all syntax errors are corrected then only program execution will be started.

Eg 2:

```
print "Hello"
```

SyntaxError: Missing parentheses in call to 'print'

Exception Handling

2. Runtime Errors:

Also known as exceptions.

While executing the program if something goes wrong because of end user input or programming logic or memory problems etc then we will get Runtime Errors.

Eg: `print(10/0)` ==> `ZeroDivisionError: division by zero`

`print(10/"ten")` ==> `TypeError: unsupported operand type(s) for /: 'int' and 'str'`

```
x=int(input("Enter Number:"))  
print(x)
```

Note: Exception Handling concept applicable for Runtime Errors but not for syntax errors

Exception Handling

What is Exception:

An unwanted and unexpected event that disturbs normal flow of program is called exception.

Eg:

ZeroDivisionError

TypeError

ValueError

FileNotFoundError

EOFError

SleepingError

TyrePuncturedError

It is highly recommended to handle exceptions. The main objective of exception handling is Graceful Termination of the program(i.e we should not block our resources and we should not miss anything)

Exception handling does not mean repairing exception. We have to define alternative way to continue rest of the program normally.

Exception Handling

Default Exception Handling in Python:

Every exception in Python is an object. For every exception type the corresponding classes are available.

Whenever an exception occurs PVM will create the corresponding exception object and will check for handling code. If handling code is not available then Python interpreter terminates the program abnormally and prints corresponding exception information to the console.

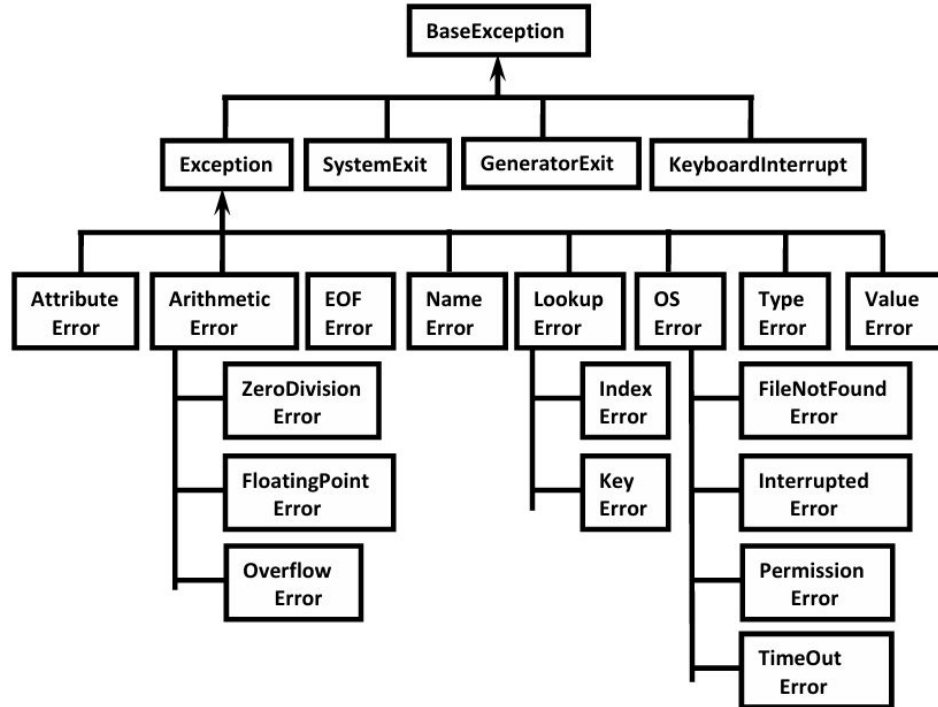
The rest of the program won't be executed.

Eg:

```
1) print("Hello")
2) print(10/0)
3) print("Hi")
4)
5) D:\Python_classes>py test.py
6) Hello
7) Traceback (most recent call last):
8)   File "test.py", line 2, in <module>
9)     print(10/0)
10) ZeroDivisionError: division by zero
```

Exception Handling

Python's Exception Hierarchy



Exception Handling

Every Exception in Python is a class.

All exception classes are child classes of BaseException.i.e every exception class extends BaseException either directly or indirectly. Hence BaseException acts as root for Python Exception Hierarchy.

Most of the times being a programmer we have to concentrate Exception and its child classes.

Exception Handling

Customized Exception Handling by using try-except:

It is highly recommended to handle exceptions.

The code which may raise exception is called risky code and we have to take risky code inside try block. The corresponding handling code we have to take inside except block.

try:

Risky Code

except XXX:

Handling code/Alternative Code

Exception Handling

without try-except:

```
1. print("stmt-1")
2. print(10/0)
3. print("stmt-3")
4.
5. Output
6. stmt-1
7. ZeroDivisionError: division by zero
```

Abnormal termination/Non-Graceful Termination

with try-except:

```
1. print("stmt-1")
2. try:
3.     print(10/0)
4. except ZeroDivisionError:
5.     print(10/2)
6. print("stmt-3")
7.
8. Output
9. stmt-1
10. 5.0
11. stmt-3
```

Normal termination/Graceful Termination

Exception Handling

Control Flow in try-except:

```
try:  
    stmt-1  
    stmt-2  
    stmt-3  
except XXX:  
    stmt-4  
stmt-5
```

case-1: If there is no exception

1,2,3,5 and Normal Termination

Exception Handling

case-2: If an exception raised at stmt-2 and corresponding except block matched
1,4,5 Normal Termination

case-3: If an exception raised at stmt-2 and corresponding except block not matched
1, Abnormal Termination

case-4: If an exception raised at stmt-4 or at stmt-5 then it is always abnormal termination.

Conclusions:

1. within the try block if anywhere exception raised then rest of the try block wont be executed eventhough we handled that exception. Hence we have to take only risky code inside try block and length of the try block should be as less as possible.
2. In addition to try block,there may be a chance of raising exceptions inside except and finally blocks also.
3. If any statement which is not part of try block raises an exception then it is always abnormal termination.

Exception Handling

How to print exception information:

try:

1. `print(10/0)`
2. `except ZeroDivisionError as msg:`
3. `print("exception raised and its description is:",msg)`
- 4.
5. Output exception raised **and** its description **is**: division by zero

Exception Handling

try with multiple except blocks:

The way of handling exception is varied from exception to exception. Hence for every exception type a separate except block we have to provide. i.e try with multiple except blocks is possible and recommended to use.

Eg:

try:

except ZeroDivisionError:

 perform alternative
 arithmetic operations

except FileNotFoundError:

 use local file instead of remote file

If try with multiple except blocks available then based on raised exception the corresponding except block will be executed.

Exception Handling

```
try:
```

```
    x=int(input("Enter First Number: "))
```

```
    y=int(input("Enter Second Number: "))
```

```
    print(x/y)
```

```
except ZeroDivisionError :
```

```
    print("Can't Divide with Zero")
```

```
except ValueError:
```

```
    print("please provide int value only")
```

Exception Handling

If try with multiple except blocks available then the order of these except blocks is important .Python interpreter will always consider from top to bottom until matched except block identified.

Eg:

```
1) try:
2)     x=int(input("Enter First Number: "))
3)     y=int(input("Enter Second Number: "))
4)     print(x/y)
5) except ArithmeticError :
6)     print("ArithmeticError")
7) except ZeroDivisionError:
8)     print("ZeroDivisionError")
```

Exception Handling

Single except block that can handle multiple exceptions:

We can write a single except block that can handle multiple different types of exceptions.

```
except (Exception1,Exception2,exception3,..): or  
except (Exception1,Exception2,exception3,..) as msg :
```

Parenthesis are mandatory and this group of exceptions internally considered as tuple.

Eg:

```
1) try:  
2)     x=int(input("Enter First Number: "))  
3)     y=int(input("Enter Second Number: "))  
4)     print(x/y)  
5) except (ZeroDivisionError,ValueError) as msg:  
6)     print("Plz Provide valid numbers only and problem is: ",msg)  
7)  
8) D:\Python_classes>py test.py  
9) Enter First Number: 10  
10) Enter Second Number: 0  
11) Plz Provide valid numbers only and problem is: division by zero  
12)  
13) D:\Python_classes>py test.py  
14) Enter First Number: 10  
15) Enter Second Number: ten  
16) Plz Provide valid numbers only and problem is: invalid literal for int() with b  
17) ase 10: 'ten'
```


Exception Handling

Default except block:

We can use default except block to handle any type of exceptions.
In default except block generally we can print normal error messages.

Syntax:

```
except:  
    statements
```

Eg:

```
1) try:  
2)     x=int(input("Enter First Number: "))  
3)     y=int(input("Enter Second Number: "))  
4)     print(x/y)  
5) except ZeroDivisionError:  
6)     print("ZeroDivisionError:Can't divide with zero")  
7) except:  
8)     print("Default Except:Plz provide valid input only")
```

Exception Handling

*****Note:** If try with multiple except blocks available then default except block should be last, otherwise we will get SyntaxError.

Eg:

```
1) try:  
2)     print(10/0)  
3) except:  
4)     print("Default Except")  
5) except ZeroDivisionError:  
6)     print("ZeroDivisionError")  
7)  
8) SyntaxError: default 'except:' must be last
```

Exception Handling

Note:

The following are various possible combinations of except blocks

1. `except ZeroDivisionError:`
1. `except ZeroDivisionError as msg:`
3. `except (ZeroDivisionError, ValueError) :`
4. `except (ZeroDivisionError, ValueError) as msg:`
5. `except :`

Exception Handling

finally block:

1. It is not recommended to maintain clean up code(Resource Deallocating Code or Resource Releasing code) inside try block because there is no guarantee for the execution of every statement inside try block always.
2. It is not recommended to maintain clean up code inside except block, because if there is no exception then except block won't be executed.

Exception Handling

Hence we required some place to maintain clean up code which should be executed always irrespective of whether exception raised or not raised and whether exception handled or not handled. Such type of best place is nothing but finally block.

Hence the main purpose of finally block is to maintain clean up code.

```
try:  
    Risky Code  
except:  
    Handling Code  
finally:  
    Cleanup code
```

The speciality of finally block is it will be executed always whether exception raised or not raised and whether exception handled or not handled.

Exception Handling

Case-1: If there is no exception

```
1) try:
2)     print("try")
3) except:
4)     print("except")
5) finally:
6)     print("finally")
7)
8) Output
9) try
10) finally
```

Exception Handling

Case-2: If there is an exception raised but handled:

```
1) try:
2)     print("try")
3)     print(10/0)
4) except ZeroDivisionError:
5)     print("except")
6) finally:
7)     print("finally")
8)
9) Output
10) try
11) except
12) finally
```

Exception Handling

Case-3: If there is an exception raised but not handled:

```
1) try:
2)     print("try")
3)     print(10/0)
4) except NameError:
5)     print("except")
6) finally:
7)     print("finally")
8)
9) Output
10) try
11) finally
12) ZeroDivisionError: division by zero(Abnormal Termination)
```


Exception Handling

*** **Note:** There is only one situation where finally block won't be executed ie whenever we are using `os._exit(0)` function.

Whenever we are using `os._exit(0)` function then Python Virtual Machine itself will be shutdown. In this particular case finally won't be executed.

```
1) imports
2) try:
3)     print("try")
4)     os._exit(0)
5) except NameError:
6)     print("except")
7) finally:
8)     print("finally")
9)
10) Output
11) try
```

Note:

`os._exit(0)`

where 0 represents status code and it indicates normal termination

There are multiple status codes are possible.

Exception Handling

Control flow in try-except-finally:

try:	<u>Case-1:</u> If there is no exception 1,2,3,5,6 Normal Termination
stmt-1	
stmt-2	
stmt-3	<u>Case-2:</u> If an exception raised at stmt2 and the corresponding except block matched 1,4,5,6 Normal Termination
except:	
stmt-4	<u>Case-3:</u> If an exception raised at stmt2 but the corresponding except block not matched 1,5 Abnormal Termination
finally:	
stmt-5	
stmt6	<u>Case-4:</u> If an exception raised at stmt4 then it is always abnormal termination but before that finally block will be executed.
	<u>Case-5:</u> If an exception raised at stmt-5 or at stmt-6 then it is always abnormal termination

Exception Handling

Nested try-except-finally blocks:

We can take try-except-finally blocks inside try or except or finally blocks.i.e nesting of try-except-finally is possible.

try:

try:

except:

except:

General Risky code we have to take inside outer try block and too much risky code we have to take inside inner try block. Inside Inner try block if an exception raised then inner except block is responsible to handle. If it is unable to handle then outer except block is responsible to handle.

Exception Handling

Eg:

```
1) try:
2)     print("outer try block")
3)     try:
4)         print("Inner try block")
5)         print(10/0)
6)     except ZeroDivisionError:
7)         print("Inner except block")
8)     finally:
9)         print("Inner finally block")
10) except:
11)     print("outer except block")
12) finally:
13)     print("outer finally block")
14)
```

15) **Output**

```
16) outer try block
17) Inner try block
18) Inner except block
19) Inner finally block
20) outer finally block
```

Exception Handling

Control flow in nested try-except-finally:

try:	<u>case-1:</u> If there is no exception
stmt-1	1,2,3,4,5,6,8,9,11,12 Normal Termination
stmt-2	
stmt-3	
try:	<u>case-2:</u> If an exception raised at stmt-2 and the corresponding except block matched
stmt-4	1,10,11,12 Normal Termination
stmt-5	
stmt-6	<u>case-3:</u> If an exception raised at stmt-2 and the corresponding except block not matched
except X:	1,11, Abnormal Termination
stmt-7	
finally:	<u>case-4:</u> If an exception raised at stmt-5 and inner except block matched
stmt-8	1,2,3,4,7,8,9,11,12 Normal Termination
stmt-9	
except Y:	
stmt-10	<u>case-5:</u> If an exception raised at stmt-5 and inner except block not matched but outer
finally:	except block matched
stmt-11	
stmt-12	1,2,3,4,8,10,11,12, Normal Termination

Exception Handling

case-6: If an exception raised at stmt-5 and both inner and outer except blocks are not matched

1,2,3,4,8,11,Abnormal Termination

case-7: If an exception raised at stmt-7 and corresponding except block matched

1,2,3,,,,,8,10,11,12,Normal Termination

case-8: If an exception raised at stmt-7 and corresponding except block not matched

1,2,3,,,,,8,11,Abnormal Termination

case-9: If an exception raised at stmt-8 and corresponding except block matched

1,2,3,,,,,,10,11,12 Normal Termination

case-10: If an exception raised at stmt-8 and corresponding except block not matched

1,2,3,,,,,,11,Abnormal Termination

Exception Handling

case-11: If an exception raised at stmt-9 and corresponding except block matched
1,2,3,,,,,,8,10,11,12,Normal Termination

case-12: If an exception raised at stmt-9 and corresponding except block not matched
1,2,3,,,,,,8,11,Abnormal Termination

case-13: If an exception raised at stmt-10 then it is always abnormal termination but
before abnormal termination finally block(stmt-11) will be executed.

case-14: If an exception raised at stmt-11 or stmt-12 then it is always abnormal
termination.

Note: If the control entered into try block then compulsory finally block will be executed.
If the control not entered into try block then finally block won't be executed.

Exception Handling

else block with try-except-finally:

We can use else block with try-except-finally blocks.

else block will be executed if and only if there are no exceptions inside try block.

```
try:  
    Risky Code  
except:  
    will be executed if exception inside try  
else:  
    will be executed if there is no exception inside try  
finally:  
    will be executed whether exception raised or not raised and handled or not  
handled
```


Exception Handling

```
try:
    print("try")
    print(10/0)--->1
except:
    print("except")
else:
    print("else")
finally:
    print("finally")
```

If we comment line-1 then else block will be executed b'z there is no exception inside try.
In this case the output is:

```
try
else
finally
```

Exception Handling

Various possible combinations of try-except-else-finally:

1. Whenever we are writing try block, compulsory we should write except or finally block.i.e without except or finally block we cannot write try block.
2. Whenever we are writing except block, compulsory we should write try block. i.e except without try is always invalid.
3. Whenever we are writing finally block, compulsory we should write try block. i.e finally without try is always invalid.
4. We can write multiple except blocks for the same try,but we cannot write multiple finally blocks for the same try
5. Whenever we are writing else block compulsory except block should be there. i.e without except we cannot write else block.
6. In try-except-else-finally order is important.
7. We can define try-except-else-finally inside try,except,else and finally blocks. i.e nesting of try-except-else-finally is always possible.

Exception Handling

1	try: print("try")	✗
2	except: print("Hello")	✗
3	else: print("Hello")	✗
4	finally: print("Hello")	✗
5	try: print("try") except: print("except")	✓
6	try: print("try") finally: print("finally")	✓
7	try: print("try") except: print("except") else: print("else")	✓

Exception Handling

8	try: print("try") else: print("else")	✗
9	try: print("try") else: print("else") finally: print("finally")	✗
10	try: print("try") except XXX: print("except-1") except YYY: print("except-2")	✓
11	try: print("try") except : print("except-1") else: print("else") else: print("else")	✗

12	try: print("try") except : print("except-1") finally: print("finally") finally: print("finally")	✗
13	try: print("try") print("Hello") except: print("except")	✗
14	try: print("try") except: print("except") print("Hello") except: print("except")	✗

Exception Handling

15	<pre>try: print("try") except: print("except") print("Hello") finally: print("finally")</pre>	✗
16	<pre>try: print("try") except: print("except") print("Hello") else: print("else")</pre>	✗
17	<pre>try: print("try") except: print("except") try: print("try") except: print("except")</pre>	✓

18	<pre>try: print("try") except: print("except") try: print("try") finally: print("finally")</pre>	✓
19	<pre>try: print("try") except: print("except") if 10>20: print("if") else: print("else")</pre>	✗
20	<pre>try: print("try") try: print("inner try") except: print("inner except block") finally: print("inner finally block") except: print("except")</pre>	✓

Exception Handling

21	<pre>try: print("try") except: print("except") try: print("inner try") except: print("inner except block") finally: print("inner finally block")</pre>	✓
22	<pre>try: print("try") except: print("except") finally: try: print("inner try") except: print("inner except block") finally: print("inner finally block")</pre>	✓

23	<pre>try: print("try") except: print("except") try: print("try") else: print("else")</pre>	✗
24	<pre>try: print("try") try: print("inner try") except: print("except")</pre>	✗
25	<pre>try: print("try") else: print("else") except: print("except") finally: print("finally")</pre>	✗

Exception Handling

Types of Exceptions:

In Python there are 2 types of exceptions are possible.

1. Predefined Exceptions
2. User Defined Exceptions

Exception Handling

1. Predefined Exceptions:

Also known as in-built exceptions

The exceptions which are raised automatically by Python virtual machine whenever a particular event occurs, are called pre defined exceptions.

Eg 1: Whenever we are trying to perform Division by zero, automatically Python will raise ZeroDivisionError.

```
print(10/0)
```

Eg 2: Whenever we are trying to convert input value to int type and if input value is not int value then Python will raise ValueError automatically

```
x=int("ten")==>ValueError
```


Exception Handling

2. User Defined Exceptions:

Also known as Customized Exceptions or Programatic Exceptions

Some time we have to define and raise exceptions explicitly to indicate that something goes wrong ,such type of exceptions are called User Defined Exceptions or Customized Exceptions

Programmer is responsible to define these exceptions and Python not having any idea about these. Hence we have to raise explicitly based on our requirement by using "raise" keyword.

Eg:

InSufficientFundsException

InvalidInputException

TooYoungException

TooOldException

Exception Handling

How to Define and Raise Customized Exceptions:

Every exception in Python is a class that extends Exception class either directly or indirectly.

Syntax:

```
class classname(predefined exception class name):  
    def __init__(self,arg):  
        self.msg=arg
```

Eg:

```
1) class TooYoungException(Exception):  
2)     def __init__(self,arg):  
3)         self.msg=arg
```

TooYoungException is our class name which is the child class of Exception

Exception Handling

We can raise exception by using raise keyword as follows
`raise TooYoungException("message")`

Eg:

```
1) class TooYoungException(Exception):
2)     def __init__(self,arg):
3)         self.msg=arg
4)
5) class TooOldException(Exception):
6)     def __init__(self,arg):
7)         self.msg=arg
8)
9) age=int(input("Enter Age:"))
10) if age>60:
11)     raise TooYoungException("Plz wait some more time you will get best match soon!!!")
12) elif age<18:
13)     raise TooOldException("Your age already crossed marriage age...no chance of getting marriage")
14) else:
15)     print("You will get match details soon by email!!!")
```

Python Logging

Logging the Exceptions:

It is highly recommended to store complete application flow and exceptions information to a file. This process is called logging.

The main advantages of logging are:

1. We can use log files while performing debugging
2. We can provide statistics like number of requests per day etc

To implement logging, Python provides one inbuilt module logging.

Python Logging

logging levels:

Depending on type of information, logging data is divided according to the following 6 levels in Python.

table

1. CRITICAL==>50==>Represents a very serious problem that needs high attention
2. ERROR==>40==>Represents a serious error
3. WARNING==>30==>Represents a warning message, some caution needed. It is alert to the programmer
4. INFO==>20==>Represents a message with some important information
5. DEBUG==>10==>Represents a message with debugging information
6. NOTSET==>0==>Represents that the level is not set.

By default while executing Python program only WARNING and higher level messages will be displayed.

Python Logging

How to implement logging:

To perform logging, first we required to create a file to store messages and we have to specify which level messages we have to store.

We can do this by using `basicConfig()` function of logging module.

```
logging.basicConfig(filename='log.txt',level=logging.WARNING)
```

Python Logging

The above line will create a file log.txt and we can store either WARNING level or higher level messages to that file.

After creating log file, we can write messages to that file by using the following methods.

```
logging.debug(message)  
logging.info(message)  
logging.warning(message)  
logging.error(message)  
logging.critical(message)
```

Python Logging

Q. Write a Python program to create a log file and write WARNING and higher level messages?

```
1) import logging
2) logging.basicConfig(filename='log.txt', level=logging.WARNING)
3) print("Logging Module Demo")
4) logging.debug("This is debug message")
5) logging.info("This is info message")
6) logging.warning("This is warning message")
7) logging.error("This is error message")
8) logging.critical("This is critical message")
```

log.txt:

```
1) WARNING:root:This is warning message
2) ERROR:root:This is error message
3) CRITICAL:root:This is critical message
```


Python Logging

Note:

In the above program only WARNING and higher level messages will be written to log file. If we set level as DEBUG then all messages will be written to log file.

```
1) import logging
2) logging.basicConfig(filename='log.txt',level=logging.DEBUG)
3) print("Logging Module Demo")
4) logging.debug("This is debug message")
5) logging.info("This is info message")
6) logging.warning("This is warning message")
7) logging.error("This is error message")
8) logging.critical("This is critical message")
```

Python Logging

How to write Python program exceptions to the log file:

By using the following function we can write exceptions information to the log file.

```
logging.exception(msg)
```

Q. Python Program to write exception information to the log file

```
1) import logging
2) logging.basicConfig(filename='mylog.txt',level=logging.INFO)
3) logging.info("A New request Came:")
4) try:
5)     x=int(input("Enter First Number: "))
6)     y=int(input("Enter Second Number: "))
7)     print(x/y)
8) except ZeroDivisionError as msg:
9)     print("cannot divide with zero")
10)    logging.exception(msg)
11) except ValueError as msg:
12)    print("Enter only int values")
13)    logging.exception(msg)
14) logging.info("Request Processing Completed")
```

Python Logging

PYTHON DEBUGGING BY USING ASSERTIONS

Debugging Python Program by using assert keyword:

The process of identifying and fixing the bug is called debugging.

Very common way of debugging is to use `print()` statement. But the problem with the `print()` statement is after fixing the bug, compulsory we have to delete the extra added `print()` statements, otherwise these will be executed at runtime which creates performance problems and disturbs console output.

To overcome this problem we should go for `assert` statement. The main advantage of `assert` statement over `print()` statement is after fixing bug we are not required to delete `assert` statements. Based on our requirement we can enable or disable `assert` statements.

Hence the main purpose of assertions is to perform debugging. Usually we can perform debugging either in development or in test environments but not in production environment. Hence assertions concept is applicable only for dev and test environments but not for production environment.

Python Logging

Types of assert statements:

There are 2 types of assert statements

1. Simple Version
2. Augmented Version

1. Simple Version:

`assert conditional_expression`

2. Augmented Version:

`assert conditional_expression,message`

`conditional_expression` will be evaluated and if it is true then the program will be continued.

If it is false then the program will be terminated by raising `AssertionError`.

By seeing `AssertionError`, programmer can analyze the code and can fix the problem.

Python Logging

Eg:

```
1) def squarelt(x):  
2)     return x**x  
  
3) assert squarelt(2)==4,"The square of 2 should be 4"  
4) assert squarelt(3)==9,"The square of 3 should be 9"  
5) assert squarelt(4)==16,"The square of 4 should be 16"  
6) print(squarelt(2))  
7) print(squarelt(3))  
8) print(squarelt(4))
```

```
16) def squarelt(x):  
17)     return x*x  
18) assert squarelt(2)==4,"The square of 2 should be 4"  
19) assert squarelt(3)==9,"The square of 3 should be 9"  
20) assert squarelt(4)==16,"The square of 4 should be 16"  
21) print(squarelt(2))  
22) print(squarelt(3))  
23) print(squarelt(4))  
24)  
25) Output  
26) 4  
27) 9  
28) 16
```

Python Logging

Exception Handling vs Assertions:

Assertions concept can be used to alert programmer to resolve development time errors.

Exception Handling can be used to handle runtime errors.

Review of Python essentials and coding conventions

Object Oriented Programming(OOP) in Python

What is Class:

- ⚽ In Python every thing is an object. To create objects we required some Model or Plan or Blue print, which is nothing but class.
- ⚽ We can write a class to represent properties (attributes) and actions (behaviour) of object.
- ⚽ Properties can be represented by variables
- ⚽ Actions can be represented by Methods.
- ⚽ Hence class contains both variables and methods.

Object Oriented Programming(OOP) in Python

How to Define a class?

We can define a class by using class keyword.

Syntax:

```
class className:  
    """ documentation string """  
    variables:instance variables,static and local variables  
    methods: instance methods,static methods,class methods
```

Documentation string represents description of the class. Within the class doc string is always optional. We can get doc string by using the following 2 ways.

1. `print(classname.__doc__)`
2. `help(classname)`

Example:

```
1) class Student:  
2)     """ This is student class with required data """  
3) print(Student.__doc__)  
4) help(Student)
```

Within the Python class we can represent data by using variables.

Object Oriented Programming(OOP) in Python

Within the Python class we can represent data by using variables.

There are 3 types of variables are allowed.

- 1. Instance Variables (Object Level Variables)**
- 2. Static Variables (Class Level Variables)**
- 3. Local variables (Method Level Variables)**

Within the Python class, we can represent operations by using methods. The following are various types of allowed methods

- 1. Instance Methods**
- 2. Class Methods**
- 3. Static Methods**

Object Oriented Programming(OOP) in Python

Example for class:

```
1) class Student:
2)     """Developed by durga for python demo"""
3)     def __init__(self):
4)         self.name='durga'
5)         self.age=40
6)         self.marks=80
7)
8)     def talk(self):
9)         print("Hello I am :",self.name)
10)        print("My Age is:",self.age)
11)        print("My Marks are:",self.marks)
```

Object Oriented Programming(OOP) in Python

What is Object:

Physical existence of a class is nothing but object. We can create any number of objects for a class.

Syntax to create object: `referencevariable = classname()`

Example: `s = Student()`

What is Reference Variable:

The variable which can be used to refer object is called reference variable.

By using reference variable, we can access properties and methods of object.

Object Oriented Programming(OOP) in Python

Program: Write a Python program to create a Student class and Creates an object to it. Call the method talk() to display student details

```
1) class Student:
2)
3)     def __init__(self,name,rollno,marks):
4)         self.name=name
5)         self.rollno=rollno
6)         self.marks=marks
7)
8)     def talk(self):
9)         print("Hello My Name is:",self.name)
10)        print("My Rollno is:",self.rollno)
11)        print("My Marks are:",self.marks)
12)
13) s1=Student("Durga",101,80)
14) s1.talk()
```

Object Oriented Programming(OOP) in Python

Self variable:

self is the default variable which is always pointing to current object (like this keyword in Java)

By using self we can access instance variables and instance methods of object.

Note:

1. self should be first parameter inside constructor

```
def __init__(self):
```

2. self should be first parameter inside instance methods

```
def talk(self):
```

Object Oriented Programming(OOP) in Python

Constructor Concept:

- ☕ Constructor is a special method in python.
 - ☕ The name of the constructor should be `__init__(self)`
 - ☕ Constructor will be executed automatically at the time of object creation.
 - ☕ The main purpose of constructor is to declare and initialize instance variables.
 - ☕ Per object constructor will be executed only once.
 - ☕ Constructor can take atleast one argument(atleast self)
-
- ☕ Constructor is optional and if we are not providing any constructor then python will provide default constructor.

Example:

```
1) def __init__(self,name,rollno,marks):  
2)     self.name=name  
3)     self.rollno=rollno  
4)     self.marks=marks
```

Object Oriented Programming(OOP) in Python

Program to demonstrate constructor will execute only once per object:

```
1) class Test:
2)
3)     def __init__(self):
4)         print("Constructor exeuction...")
5)
6)     def m1(self):
7)         print("Method execution...")
8)
9) t1=Test()
10) t2=Test()
11) t3=Test()
12) t1.m1()
```


Object Oriented Programming(OOP) in Python

Program:

```
1) class Student:
2)
3)     """ This is student class with required data"""
4)     def __init__(self,x,y,z):
5)         self.name=x
6)         self.rollno=y
7)         self.marks=z
8)
9)     def display(self):
10)        print("Student Name:{}\nRollno:{} \nMarks:{}".format(self.name,self.rollno,self.marks)
11)        )
12) s1=Student("Durga",101,80)
13) s1.display()
14) s2=Student("Sunny",102,100)
15) s2.display()
```

Object Oriented Programming(OOP) in Python

Differences between Methods and Constructors:

Method	Constructor
1. Name of method can be any name	1. Constructor name should be always <code>__init__</code>
2. Method will be executed if we call that method	2. Constructor will be executed automatically at the time of object creation.
3. Per object, method can be called any number of times.	3. Per object, Constructor will be executed only once
4. Inside method we can write business logic	4. Inside Constructor we have to declare and initialize instance variables

Object Oriented Programming(OOP) in Python

Types of Variables:

Inside Python class 3 types of variables are allowed.

1. Instance Variables (Object Level Variables)
2. Static Variables (Class Level Variables)
3. Local variables (Method Level Variables)

1. Instance Variables:

If the value of a variable is varied from object to object, then such type of variables are called instance variables.

For every object a separate copy of instance variables will be created.

Where we can declare Instance variables:

1. Inside Constructor by using self variable
2. Inside Instance Method by using self variable
3. Outside of the class by using object reference variable

Object Oriented Programming(OOP) in Python

1. Inside Constructor by using self variable:

We can declare instance variables inside a constructor by using self keyword. Once we creates object, automatically these variables will be added to the object.

Example:

```
1) class Employee:
2)
3)     def __init__(self):
4)         self.eno=100
5)         self.ename='Durga'
6)         self.esal=10000
7)
8) e=Employee()
9) print(e.__dict__)
```

Object Oriented Programming(OOP) in Python

2. Inside Instance Method by using self variable:

We can also declare instance variables inside instance method by using self variable. If any instance variable declared inside instance method, that instance variable will be added once we call that method.

Example:

```
1) class Test:
2)
3)     def __init__(self):
4)         self.a=10
5)         self.b=20
6)
7)     def m1(self):
8)         self.c=30
9)
10) t=Test()
11) t.m1()
12) print(t.__dict__)
```

Object Oriented Programming(OOP) in Python

3. Outside of the class by using object reference variable:

We can also add instance variables outside of a class to a particular object.

```
1) class Test:
2)
3)     def __init__(self):
4)         self.a=10
5)         self.b=20
6)
7)     def m1(self):
8)         self.c=30
9)
10) t=Test()
11) t.m1()
12) t.d=40
13) print(t.__dict__)
```

Output {'a': 10, 'b': 20, 'c': 30, 'd': 40}

Object Oriented Programming(OOP) in Python

How to access Instance variables:

We can access instance variables with in the class by using self variable and outside of the class by using object reference.

```
1) class Test:
2)
3)     def __init__(self):
4)         self.a=10
5)         self.b=20
6)
7)     def display(self):
8)         print(self.a)
9)         print(self.b)
10)
11) t=Test()
12) t.display()
13) print(t.a,t.b)
```

Output

```
10
20
10 20
```

Object Oriented Programming(OOP) in Python

How to delete instance variable from the object:

1. Within a class we can delete instance variable as follows

```
del self.variableName
```

2. From outside of class we can delete instance variables as follows

```
del objectreference.variableName
```

Example:

```
1) class Test:
2)     def __init__(self):
3)         self.a=10
4)         self.b=20
5)         self.c=30
6)         self.d=40
7)     def m1(self):
8)         del self.d
9)
10) t=Test()
11) print(t.__dict__)
```

.....

```
.....
12) t.m1()
13) print(t.__dict__)
14) del t.c
15) print(t.__dict__)
```


Object Oriented Programming(OOP) in Python

Note: The instance variables which are deleted from one object, will not be deleted from other objects.

Example:

```
1) class Test:
2)     def __init__(self):
3)         self.a=10
4)         self.b=20
5)         self.c=30
6)         self.d=40
7)
8)
9) t1=Test()
10) t2=Test()
11) del t1.a
12) print(t1.__dict__)
13) print(t2.__dict__)
```

Object Oriented Programming(OOP) in Python

If we change the values of instance variables of one object then those changes won't be reflected to the remaining objects, because for every object we are separate copy of instance variables are available.

Example:

```
1) class Test:
2)     def __init__(self):
3)         self.a=10
4)         self.b=20
5)
6) t1=Test()
7) t1.a=888
8) t1.b=999
9) t2=Test()
10) print('t1:',t1.a,t1.b)
11) print('t2:',t2.a,t2.b)
```

Object Oriented Programming(OOP) in Python

1. Static variables:

If the value of a variable is not varied from object to object, such type of variables we have to declare with in the class directly but outside of methods. Such type of variables are called Static variables.

For total class only one copy of static variable will be created and shared by all objects of that class.

We can access static variables either by class name or by object reference. But recommended to use class name.

Object Oriented Programming(OOP) in Python

Instance Variable vs Static Variable:

Note: In the case of instance variables for every object a separate copy will be created, but in the case of static variables for total class only one copy will be created and shared by every object of that class.

```
1) class Test:
2)     x=10
3)     def __init__(self):
4)         self.y=20
5)
6) t1=Test()
7) t2=Test()
8) print('t1:',t1.x,t1.y)
9) print('t2:',t2.x,t2.y)
10) Test.x=888
11) t1.y=999
12) print('t1:',t1.x,t1.y)
13) print('t2:',t2.x,t2.y)
```

Object Oriented Programming(OOP) in Python

Various places to declare static variables:

1. In general we can declare within the class directly but from out side of any method
2. Inside constructor by using class name
3. Inside instance method by using class name
4. Inside classmethod by using either class name or cls variable
5. Inside static method by using class name

Object Oriented Programming(OOP) in Python

```
1) class Test:
2)     a=10
3)     def __init__(self):
4)         Test.b=20
5)     def m1(self):
6)         Test.c=30
7)     @classmethod
8)     def m2(cls):
9)         cls.d1=40
10)        Test.d2=400
11)     @staticmethod
12)     def m3():
13)         Test.e=50
```

```
14) print(Test.__dict__)
15) t=Test()
16) print(Test.__dict__)
17) t.m1()
18) print(Test.__dict__)
19) Test.m2()
20) print(Test.__dict__)
21) Test.m3()
22) print(Test.__dict__)
23) Test.f=60
24) print(Test.__dict__)
```

Object Oriented Programming(OOP) in Python

How to access static variables:

1. inside constructor: by using either self or classname
2. inside instance method: by using either self or classname
3. inside class method: by using either cls variable or classname
4. inside static method: by using classname
5. From outside of class: by using either object reference or classnae

```
1) class Test:
2)     a=10
3)     def __init__(self):
4)         print(self.a)
5)         print(Test.a)
6)     def m1(self):
7)         print(self.a)
```

```
8)         print(Test.a)
9)     @classmethod
10)    def m2(cls):
11)        print(cls.a)
12)        print(Test.a)
13)    @staticmethod
14)    def m3():
15)        print(Test.a)
16) t=Test()
17) print(Test.a)
18) print(t.a)
19) t.m1()
20) t.m2()
21) t.m3()
```

Object Oriented Programming(OOP) in Python

Where we can modify the value of static variable:

Anywhere either with in the class or outside of class we can modify by using classname.
But inside class method, by using cls variable.

Example:

```
1) class Test:
2)     a=777
3)     @classmethod
4)     def m1(cls):
5)         cls.a=888
6)     @staticmethod
7)     def m2():
8)         Test.a=999
9) print(Test.a)
10) Test.m1()
11) print(Test.a)
12) Test.m2()
13) print(Test.a)
```


Object Oriented Programming(OOP) in Python

If we change the value of static variable by using either self or object reference variable:

If we change the value of static variable by using either self or object reference variable, then the value of static variable won't be changed, just a new instance variable with that name will be added to that particular object.

Example 1:

```
1) class Test:
2)     a=10
3)     def m1(self):
4)         self.a=888
5) t1=Test()
6) t1.m1()
7) print(Test.a)
8) print(t1.a)
```

Object Oriented Programming(OOP) in Python

Example:

```
1) class Test:
2)     x=10
3)     def __init__(self):
4)         self.y=20
5)
6) t1=Test()
7) t2=Test()
8) print('t1:',t1.x,t1.y)
9) print('t2:',t2.x,t2.y)
10) t1.x=888
11) t1.y=999
12) print('t1:',t1.x,t1.y)
13) print('t2:',t2.x,t2.y)
```

Object Oriented Programming(OOP) in Python

Example:

```
1) class Test:  
2)     a=10  
3)     def __init__(self):  
4)         self.b=20  
5) t1=Test()  
6) t2=Test()  
7) Test.a=888  
8) t1.b=999  
9) print(t1.a,t1.b)  
10) print(t2.a,t2.b)
```

Object Oriented Programming(OOP) in Python

```
1) class Test:  
2)     a=10  
3)     def __init__(self):  
4)         self.b=20  
5)     def m1(self):  
6)         self.a=888  
7)         self.b=999  
8)  
9) t1=Test()  
10) t2=Test()  
11) t1.m1()  
12) print(t1.a,t1.b)  
13) print(t2.a,t2.b)
```

Object Oriented Programming(OOP) in Python

```
1) class Test:
2)     a=10
3)     def __init__(self):
4)         self.b=20
5)     @classmethod
6)     def m1(cls):
7)         cls.a=888
8)         cls.b=999
9)
10) t1=Test()
11) t2=Test()
12) t1.m1()
13) print(t1.a,t1.b)
14) print(t2.a,t2.b)
15) print(Test.a,Test.b)
```

Object Oriented Programming(OOP) in Python

How to delete static variables of a class:

We can delete static variables from anywhere by using the following syntax

```
del classname.variablename
```

But inside classmethod we can also use cls variable

```
del cls.variablename
```

```
1) class Test:
2)     a=10
3)     @classmethod
4)     def m1(cls):
5)         del cls.a
6) Test.m1()
7) print(Test.__dict__)
```

Object Oriented Programming(OOP) in Python

Example:

```
1) class Test:
2)     a=10
3)     def __init__(self):
4)         Test.b=20
5)         del Test.a
6)     def m1(self):
7)         Test.c=30
8)         del Test.b
9)     @classmethod
10)    def m2(cls):
11)        cls.d=40
12)        del Test.c
13)    @staticmethod
14)    def m3():
15)        Test.e=50
16)        del Test.d
17) print(Test.__dict__)
18) t=Test()
```

```
19) print(Test.__dict__)
20) t.m1()
21) print(Test.__dict__)
22) Test.m2()
23) print(Test.__dict__)
24) Test.m3()
25) print(Test.__dict__)
26) Test.f=60
27) print(Test.__dict__)
28) del Test.e
29) print(Test.__dict__)
```

Object Oriented Programming(OOP) in Python

Note: By using object reference variable/self we can read static variables, but we cannot modify or delete.

If we are trying to modify, then a new instance variable will be added to that particular object.
t1.a = 70

If we are trying to delete then we will get error.

Example:

```
1) class Test:  
2)     a=10  
3)  
4) t1=Test()  
5) del t1.a    ==>AttributeError: a
```


Object Oriented Programming(OOP) in Python

We can modify or delete static variables only by using classname or cls variable.

```
1) import sys
2) class Customer:
3)     """ Customer class with bank operations.. """
4)     bankname='DURGABANK'
5)     def __init__(self,name,balance=0.0):
6)         self.name=name
7)         self.balance=balance
8)     def deposit(self,amt):
9)         self.balance=self.balance+amt
10)        print('Balance after deposit:',self.balance)
11)    def withdraw(self,amt):
12)        if amt>self.balance:
13)            print('Insufficient Funds..cannot perform this operation')
14)            sys.exit()
15)        self.balance=self.balance-amt
16)        print('Balance after withdraw:',self.balance)
17)
18) print('Welcome to',Customer.bankname)
```

Object Oriented Programming(OOP) in Python

```
19) name=input('Enter Your Name:')
20) c=Customer(name)
21) while True:
22)     print('d-Deposit \nw-Withdraw \ne-exit')
23)     option=input('Choose your option:')
24)     if option=='d' or option=='D':
25)         amt=float(input('Enter amount:'))
26)         c.deposit(amt)
27)     elif option=='w' or option=='W':
28)         amt=float(input('Enter amount:'))
29)         c.withdraw(amt)
30)     elif option=='e' or option=='E':
31)         print('Thanks for Banking')
32)         sys.exit()
33)     else:
34)         print('Invalid option..Plz choose valid option')
```

Object Oriented Programming(OOP) in Python

Local variables:

Sometimes to meet temporary requirements of programmer,we can declare variables inside a method directly,such type of variables are called local variable or temporary variables.

Local variables will be created at the time of method execution and destroyed once method completes.

Local variables of a method cannot be accessed from outside of method.

Example:

```
1) class Test:
2)     def m1(self):
3)         a=1000
4)         print(a)
5)     def m2(self):
6)         b=2000
7)         print(b)
8) t=Test()
9) t.m1()
10) t.m2()
```

Object Oriented Programming(OOP) in Python

Example 2:

```
1) class Test:
2)     def m1(self):
3)         a=1000
4)         print(a)
5)     def m2(self):
6)         b=2000
7)         print(a) #NameError: name 'a' is not defined
8)         print(b)
9) t=Test()
10) t.m1()
11) t.m2()
```

Object Oriented Programming(OOP) in Python

Types of Methods:

Inside Python class 3 types of methods are allowed

1. Instance Methods
2. Class Methods
3. Static Methods

Object Oriented Programming(OOP) in Python

1. Instance Methods:

Inside method implementation if we are using instance variables then such type of methods are called instance methods.

Inside instance method declaration, we have to pass self variable.

```
def m1(self):
```

By using self variable inside method we can able to access instance variables.

Within the class we can call instance method by using self variable and from outside of the class we can call by using object reference.

Object Oriented Programming(OOP) in Python

```
1) class Student:
2)     def __init__(self,name,marks):
3)         self.name=name
4)         self.marks=marks
5)     def display(self):
6)         print('Hi',self.name)
7)         print('Your Marks are:',self.marks)
8)     def grade(self):
9)         if self.marks>=60:
10)            print('You got First Grade')
11)         elif self.marks>=50:
12)            print('Yout got Second Grade')
13)         elif self.marks>=35:
14)            print('You got Third Grade')
15)         else:
16)            print('You are Failed')
17) n=int(input('Enter number of students:'))
18) for i in range(n):
19)     name=input('Enter Name:')
20)     marks=int(input('Enter Marks:'))
21)     s= Student(name,marks)
22)     s.display()
23)     s.grade()
24)     print()
```

Object Oriented Programming(OOP) in Python

Setter and Getter Methods:

We can set and get the values of instance variables by using getter and setter methods.

Setter Method:

setter methods can be used to set values to the instance variables. setter methods also known as mutator methods.

syntax:

```
def setVariable(self,variable):  
    self.variable=variable
```

Example:

```
def setName(self,name):  
    self.name=name
```


Object Oriented Programming(OOP) in Python

Getter Method:

Getter methods can be used to get values of the instance variables. Getter methods also known as accessor methods.

syntax:

```
def getVariable(self):  
    return self.variable
```

Example:

```
def getName(self):  
    return self.name
```

Object Oriented Programming(OOP) in Python

Demo Program:

```
1) class Student:
2)     def setName(self,name):
3)         self.name=name
4)
5)     def getName(self):
6)         return self.name
7)
8)     def setMarks(self,marks):
9)         self.marks=marks
10)
11)    def getMarks(self):
12)        return self.marks
13)
14) n=int(input('Enter number of students:'))
15) for i in range(n):
16)     s=Student()
17)     name=input('Enter Name:')
18)     s.setName(name)
19)     marks=int(input('Enter Marks:'))
20)     s.setMarks(marks)
21)
22)     print('Hi',s.getName())
23)     print('Your Marks are:',s.getMarks())
24)     print()
```

Object Oriented Programming(OOP) in Python

2. Class Methods:

Inside method implementation if we are using only class variables (static variables), then such type of methods we should declare as class method.

We can declare class method explicitly by using `@classmethod` decorator.
For class method we should provide `cls` variable at the time of declaration

Object Oriented Programming(OOP) in Python

We can call classmethod by using classname or object reference variable.

Demo Program:

```
1) class Animal:  
2)     legs=4  
3)     @classmethod  
4)     def walk(cls,name):  
5)         print('{} walks with {} legs...'.format(name,cls.legs))  
6) Animal.walk('Dog')  
7) Animal.walk('Cat')
```

Object Oriented Programming(OOP) in Python

Program to track the number of objects created for a class:

```
1) class Test:
2)     count=0
3)     def __init__(self):
4)         Test.count =Test.count+1
5)     @classmethod
6)     def noOfObjects(cls):
7)         print('The number of objects created for test class:',cls.count)
8)
9) t1=Test()
10) t2=Test()
11) Test.noOfObjects()
12) t3=Test()
13) t4=Test()
14) t5=Test()
15) Test.noOfObjects()
```

Object Oriented Programming(OOP) in Python

3. Static Methods:

In general these methods are general utility methods.

Inside these methods we won't use any instance or class variables.

Here we won't provide self or cls arguments at the time of declaration.

We can declare static method explicitly by using @staticmethod decorator

We can access static methods by using classname or object reference

```
1) class DurgaMath:
2)
3)     @staticmethod
4)     def add(x,y):
5)         print('The Sum:',x+y)
6)
7)     @staticmethod
8)     def product(x,y):
9)         print('The Product:',x*y)
10)
11)    @staticmethod
12)    def average(x,y):
13)        print('The average:',(x+y)/2)
14)
15) DurgaMath.add(10,20)
16) DurgaMath.product(10,20)
17) DurgaMath.average(10,20)
```

Object Oriented Programming(OOP) in Python

Note: In general we can use only instance and static methods. Inside static method we can access class level variables by using class name.

class methods are most rarely used methods in python.

Object Oriented Programming(OOP) in Python

Passing members of one class to another class:

We can access members of one class inside another class.

```
1) class Employee:
2)     def __init__(self,eno,ename,esal):
3)         self.eno=eno
4)         self.ename=ename
5)         self.esal=esal
6)     def display(self):
7)         print('Employee Number:',self.eno)
8)         print('Employee Name:',self.ename)
9)         print('Employee Salary:',self.esal)
10) class Test:
11)     def modify(emp):
12)         emp.esal=emp.esal+10000
13)         emp.display()
14) e=Employee(100,'Durga',10000)
15) Test.modify(e)
```


Object Oriented Programming(OOP) in Python

Inner classes:

Sometimes we can declare a class inside another class, such type of classes are called inner classes.

Without existing one type of object if there is no chance of existing another type of object, then we should go for inner classes.

Example: Without existing Car object there is no chance of existing Engine object. Hence Engine class should be part of Car class.

```
class Car:
    ....
    class Engine:
        .....
```

Example: Without existing university object there is no chance of existing Department object

```
class University:
    ....
    class Department:
        .....
```

Object Oriented Programming(OOP) in Python

eg3:

Without existing Human there is no chance of existin Head. Hence Head should be part of Human.

```
class Human:  
    class Head:
```

Note: Without existing outer class object there is no chance of existing inner class object. Hence inner class object is always associated with outer class object.

Demo Program-1:

```
1) class Outer:  
2)     def __init__(self):  
3)         print("outer class object creation")  
4)     class Inner:  
5)         def __init__(self):  
6)             print("inner class object creation")  
7)         def m1(self):  
8)             print("inner class method")  
9) o=Outer()  
10) i=o.Inner()  
11) i.m1()
```

Object Oriented Programming(OOP) in Python

Note: The following are various possible syntaxes for calling inner class method

1.

```
o=Outer()  
i=o.Inner()  
i.m1()
```

2.

```
i=Outer().Inner()  
i.m1()
```

3. **Outer().Inner().m1()**

Object Oriented Programming(OOP) in Python

Demo Program-2:

```
1) class Person:
2)     def __init__(self):
3)         self.name='durga'
4)         self.db=self.Dob()
5)     def display(self):
6)         print('Name:',self.name)
7)     class Dob:
8)         def __init__(self):
9)             self.dd=10
10)            self.mm=5
11)            self.yy=1947
12)        def display(self):
13)            print('Dob={}/{}/{}'.format(self.dd,self.mm,self.yy))
14) p=Person()
15) p.display()
16) x=p.db
17) x.display()
```

Object Oriented Programming(OOP) in Python

Demo Program-3:

Inside a class we can declare any number of inner classes.

```
1) class Human:
2)
3)     def __init__(self):
4)         self.name = 'Sunny'
5)         self.head = self.Head()
6)         self.brain = self.Brain()
7)     def display(self):
8)         print("Hello..",self.name)
9)
10)    class Head:
11)        def talk(self):
12)            print('Talking...')
13)
14)    class Brain:
15)        def think(self):
16)            print('Thinking...')
17)
18) h=Human()
19) h.display()
20) h.head.talk()
21) h.brain.think()
```

Object Oriented Programming(OOP) in Python

Garbage Collection:

In old languages like C++, programmer is responsible for both creation and destruction of objects. Usually programmer taking very much care while creating object, but neglecting destruction of useless objects. Because of his neglectance, total memory can be filled with useless objects which creates memory problems and total application will be down with Out of memory error.

But in Python, We have some assistant which is always running in the background to destroy useless objects. Because this assistant the chance of failing Python program with memory problems is very less. This assistant is nothing but Garbage Collector.

Hence the main objective of Garbage Collector is to destroy useless objects.

If an object does not have any reference variable then that object eligible for Garbage Collection.

Object Oriented Programming(OOP) in Python

How to enable and disable Garbage Collector in our program:

By default Gargbage collector is enabled, but we can disable based on our requirement. In this context we can use the following functions of gc module.

1. gc.isenabled()

Returns True if GC enabled

2. gc.disable()

To disable GC explicitly

3. gc.enable()

To enable GC explicitly

Example:

```
1) import gc
2) print(gc.isenabled())
3) gc.disable()
4) print(gc.isenabled())
5) gc.enable()
6) print(gc.isenabled())
```

Object Oriented Programming(OOP) in Python

Destructors:

Destructor is a special method and the name should be `__del__`

Just before destroying an object Garbage Collector always calls destructor to perform clean up activities (Resource deallocation activities like close database connection etc).

Once destructor execution completed then Garbage Collector automatically destroys that object.

Note: The job of destructor is not to destroy object and it is just to perform clean up activities.

Object Oriented Programming(OOP) in Python

Example:

```
1) import time
2) class Test:
3)     def __init__(self):
4)         print("Object Initialization...")
5)     def __del__(self):
6)         print("Fulfilling Last Wish and performing clean up activities...")
7)
8) t1=Test()
9) t1=None
10) time.sleep(5)
11) print("End of application")
```

Object Oriented Programming(OOP) in Python

Note:

If the object does not contain any reference variable then only it is eligible for GC. ie if the reference count is zero then only object eligible for GC

Example:

```
1) import time
2) class Test:
3)     def __init__(self):
4)         print("Constructor Execution...")
5)     def __del__(self):
6)         print("Destructor Execution...")
7)
8) t1=Test()
9) t2=t1
10) t3=t2
11) del t1
12) time.sleep(5)
13) print("object not yet destroyed after deleting t1")
14) del t2
15) time.sleep(5)
16) print("object not yet destroyed even after deleting t2")
17) print("I am trying to delete last reference variable...")
18) del t3
```

Object Oriented Programming(OOP) in Python

Example:

```
1) import time
2) class Test:
3)     def __init__(self):
4)         print("Constructor Execution...")
5)     def __del__(self):
6)         print("Destructor Execution...")
7)
8) list=[Test(),Test(),Test()]
9) del list
10) time.sleep(5)
11) print("End of application")
```

Object Oriented Programming(OOP) in Python

How to find the number of references of an object:

sys module contains getrefcount() function for this purpose.

```
sys.getrefcount(objectreference)
```

Example:

```
1) import sys
2) class Test:
3)     pass
4) t1=Test()
5) t2=t1
6) t3=t1
7) t4=t1
8) print(sys.getrefcount(t1))
```

Output 5

Note: For every object, Python internally maintains one default reference variable self.

Object Oriented Programming(OOP) in Python

Using members of one class inside another class:

We can use members of one class inside another class by using the following ways

1. By Composition (Has-A Relationship)
2. By Inheritance (IS-A Relationship)

Object Oriented Programming(OOP) in Python

1. By Composition (Has-A Relationship):

By using Class Name or by creating object we can access members of one class inside another class is nothing but composition (Has-A Relationship).

The main advantage of Has-A Relationship is Code Reusability.

Demo Program-1:

```
1) class Engine:
2)     a=10
3)     def __init__(self):
4)         self.b=20
5)     def m1(self):
6)         print('Engine Specific Functionality')
7) class Car:
8)     def __init__(self):
9)         self.engine=Engine()
10)    def m2(self):
11)        print('Car using Engine Class Functionality')
12)        print(self.engine.a)
13)        print(self.engine.b)
14)        self.engine.m1()
15) c=Car()
16) c.m2()
```

Object Oriented Programming(OOP) in Python

Demo Program-2:

```
1) class Car:
2)     def __init__(self,name,model,color):
3)         self.name=name
4)         self.model=model
5)         self.color=color
6)     def getinfo(self):
7)         print("Car Name:{}, Model:{} and Color:{}".format(self.name,self.model,self.color))
8)
9) class Employee:
10)     def __init__(self,ename,eno,car):
11)         self.ename=ename
12)         self.eno=eno
13)         self.car=car
14)     def empinfo(self):
15)         print("Employee Name:",self.ename)
16)         print("Employee Number:",self.eno)
17)         print("Employee Car Info:")
18)         self.car.getinfo()
19) c=Car("Innova","2.5V","Grey")
20) e=Employee('Durga',10000,c)
21) e.empinfo()
```

Object Oriented Programming(OOP) in Python

In the above program Employee class Has-A Car reference and hence Employee class can access all members of Car class.

Demo Program-3:

```
1) class X:
2)     a=10
3)     def __init__(self):
4)         self.b=20
5)     def m1(self):
6)         print("m1 method of X class")
7) class Y:
8)     c=30
9)     def __init__(self):
10)         self.d=40
11)     def m2(self):
12)         print("m2 method of Y class")
13)     def m3(self):
14)         x1=X()
15)         print(x1.a)
16)         print(x1.b)
17)         x1.m1()
18)         print(Y.c)
19)         print(self.d)
20)         self.m2()
21)         print("m3 method of Y class")
22) y1=Y()
23) y1.m3()
```


Object Oriented Programming(OOP) in Python

2. By Inheritance(IS-A Relationship):

What ever variables, methods and constructors available in the parent class by default available to the child classes and we are not required to rewrite. Hence the main advantage of inheritance is Code Reusability and we can extend existing functionality with some more extra functionality.

Syntax :

```
class childclass(parentclass):
```

Object Oriented Programming(OOP) in Python

Demo Program for inheritance:

```
1) class P:
2)     a=10
3)     def __init__(self):
4)         self.b=10
5)     def m1(self):
6)         print('Parent instance method')
7)     @classmethod
8)     def m2(cls):
9)         print('Parent class method')
10)    @staticmethod
11)    def m3():
12)        print('Parent static method')
13)
14) class C(P):
15)     pass
16)
17) c=C()
18) print(c.a)
19) print(c.b)
20) c.m1()
21) c.m2()
22) c.m3()
```

Object Oriented Programming(OOP) in Python

Eg:

- 1) `class P:`
- 2) **10 methods**
- 3) `class C(P):`
- 4) **5 methods**

In the above example Parent class contains 10 methods and these methods automatically available to the child class and we are not required to rewrite those methods(Code Reusability)
Hence child class contains 15 methods.

Object Oriented Programming(OOP) in Python

Note:

What ever members present in Parent class are by default available to the child class through inheritance.

Demo Program:

```
1) class P:  
2)     def m1(self):  
3)         print("Parent class method")  
4) class C(P):  
5)     def m2(self):  
6)         print("Child class method")  
7)  
8) c=C();  
9) c.m1()  
10) c.m2()
```

Output:

Parent class method

Child class method

Object Oriented Programming(OOP) in Python

Demo program for inheritance:

```
1) class Person:
2)     def __init__(self,name,age):
3)         self.name=name
4)         self.age=age
5)     def eatndrink(self):
6)         print('Eat Biryani and Drink Beer')
7)
8) class Employee(Person):
9)     def __init__(self,name,age,eno,esal):
10)         super().__init__(name,age)
11)         self.eno=eno
12)         self.esal=esal
13)
14)     def work(self):
15)         print("Coding Python is very easy just like drinking Chilled Beer")
16)     def empinfo(self):
17)         print("Employee Name:",self.name)
18)         print("Employee Age:",self.age)
19)         print("Employee Number:",self.eno)
20)         print("Employee Salary:",self.esal)
21)
22) e=Employee('Durga', 48, 100, 10000)
23) e.eatndrink()
24) e.work()
25) e.empinfo()
```

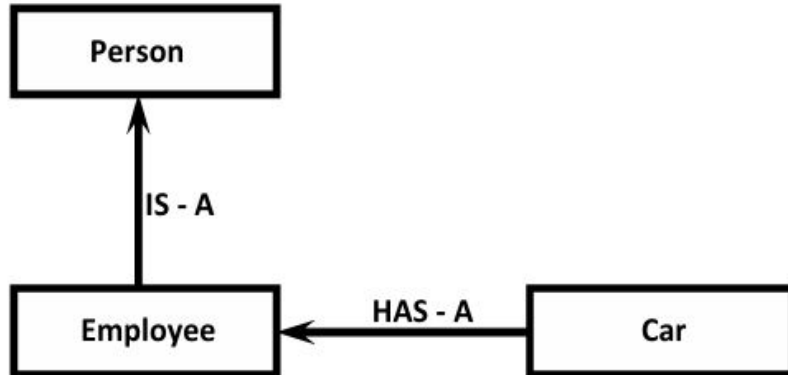
Object Oriented Programming(OOP) in Python

IS-A vs HAS-A Relationship:

If we want to extend existing functionality with some more extra functionality then we should go for IS-A Relationship

If we dont want to extend and just we have to use existing functionality then we should go for HAS-A Relationship

Eg: Employee class extends Person class Functionality
But Employee class just uses Car functionality but not extending



Object Oriented Programming(OOP) in Python

```
1) class Car:
2)     def __init__(self,name,model,color):
3)         self.name=name
4)         self.model=model
5)         self.color=color
6)     def getinfo(self):
7)         print("\tCar Name:{} \n\t Model:{} \n\t Color:{}".format(self.name,self.model,self.col
or))
8)
9) class Person:
10)     def __init__(self,name,age):
11)         self.name=name
12)         self.age=age
13)     def eatndrink(self):
14)         print('Eat Biryani and Drink Beer')
15)
```

.....Contd

Object Oriented Programming(OOP) in Python

```
16) class Employee(Person):
17)     def __init__(self,name,age,eno,esal,car):
18)         super().__init__(name,age)
19)         self.eno=eno
20)         self.esal=esal
21)         self.car=car
22)     def work(self):
23)         print("Coding Python is very easy just like drinking Chilled Beer")
24)     def empinfo(self):
25)         print("Employee Name:",self.name)
26)         print("Employee Age:",self.age)
27)         print("Employee Number:",self.eno)
28)         print("Employee Salary:",self.esal)
29)         print("Employee Car Info:")
30)         self.car.getinfo()
```

```
32) c=Car("Innova","2.5V","Grey")
33) e=Employee('Durga',48,100,10000,c)
34) e.eatndrink()
35) e.work()
36) e.empinfo()
```


Object Oriented Programming(OOP) in Python

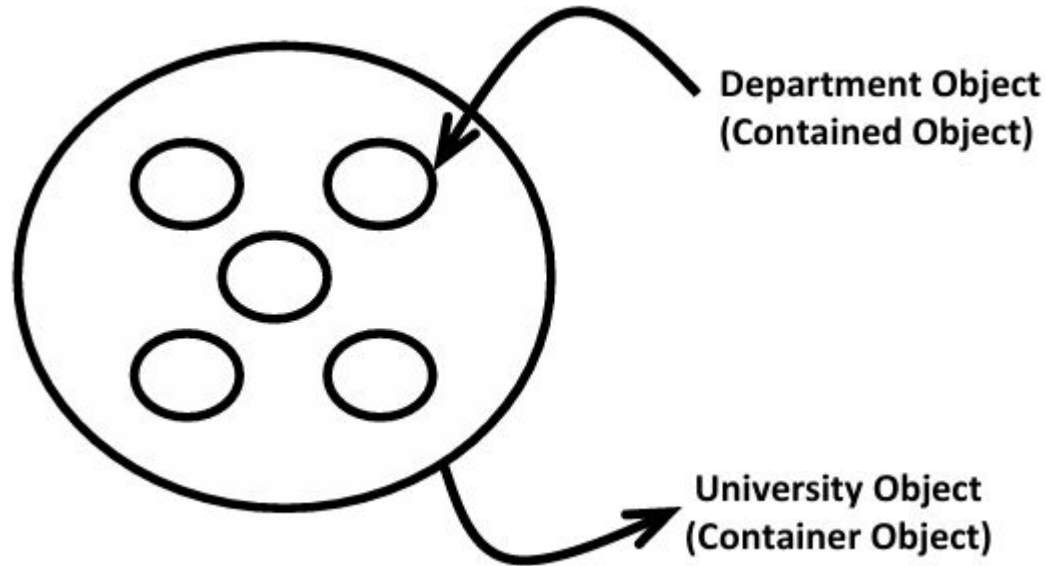
Composition vs Aggregation:

Composition:

Without existing container object if there is no chance of existing contained object then the container and contained objects are strongly associated and that strong association is nothing but Composition.

Eg: University contains several Departments and without existing university object there is no chance of existing Department object. Hence University and Department objects are strongly associated and this strong association is nothing but Composition.

Object Oriented Programming(OOP) in Python

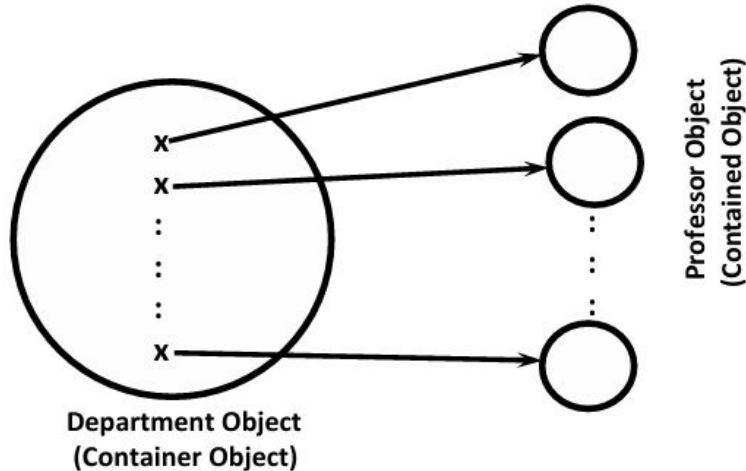


Object Oriented Programming(OOP) in Python

Aggregation:

Without existing container object if there is a chance of existing contained object then the container and contained objects are weakly associated and that weak association is nothing but Aggregation.

Eg: Department contains several Professors. Without existing Department still there may be a chance of existing Professor. Hence Department and Professor objects are weakly associated, which is nothing but Aggregation.



Object Oriented Programming(OOP) in Python

Coding Example:

```
1) class Student:  
2)     collegeName='DURGASOFT'  
3)     def __init__(self,name):  
4)         self.name=name  
5) print(Student.collegeName)  
6) s=Student('Durga')  
7) print(s.name)
```

Object Oriented Programming(OOP) in Python

In the above example without existing Student object there is no chance of existing his name. Hence Student Object and his name are strongly associated which is nothing but Composition.

But without existing Student object there may be a chance of existing collegeName. Hence Student object and collegeName are weakly associated which is nothing but Aggregation.

Object Oriented Programming(OOP) in Python

Conclusion:

The relation between object and its instance variables is always Composition where as the relation between object and static variables is Aggregation.

Note: Whenever we are creating child class object then child class constructor will be executed. If the child class does not contain constructor then parent class constructor will be executed, but parent object won't be created.

Eg:

```
1) class P:
2)     def __init__(self):
3)         print(id(self))
4) class C(P):
5)     pass
6) c=C()
7) print(id(c))
```

Object Oriented Programming(OOP) in Python

Eg:

```
1) class Person:
2)     def __init__(self,name,age):
3)         self.name=name
4)         self.age=age
5) class Student(Person):
6)     def __init__(self,name,age,rollno,marks):
7)         super().__init__(name,age)
8)         self.rollno=rollno
9)         self.marks=marks
10)    def __str__(self):
11)        return 'Name={}\nAge={}\nRollno={}\nMarks={}'.format(self.name,self.age,self.rollno
    ,self.marks)
12) s1=Student('durga',48,101,90)
13) print(s1)
```

Note: In the above example when ever we are creating child class object both parent and child class constructors got executed to perform initialization of child object

Object Oriented Programming(OOP) in Python

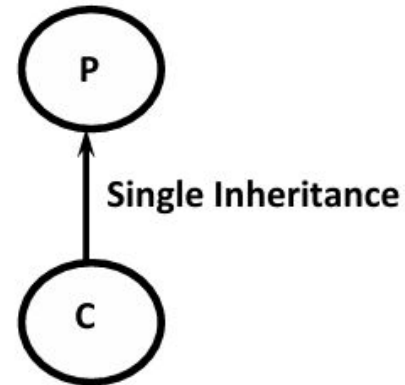
Types of Inheritance:

1. Single Inheritance:

The concept of inheriting the properties from one class to another class is known as single inheritance.

Eg:

```
1) class P:  
2)     def m1(self):  
3)         print("Parent Method")  
4) class C(P):  
5)     def m2(self):  
6)         print("Child Method")  
7) c=C()  
8) c.m1()  
9) c.m2()
```



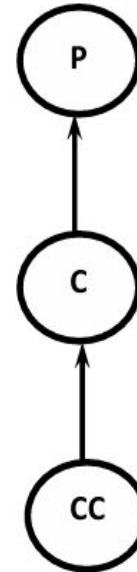
Object Oriented Programming(OOP) in Python

2. Multi Level Inheritance:

The concept of inheriting the properties from multiple classes to single class with the concept of one after another is known as multilevel inheritance

Eg:

```
1) class P:  
2)     def m1(self):  
3)         print("Parent Method")  
4) class C(P):  
5)     def m2(self):  
6)         print("Child Method")  
7) class CC(C):  
8)     def m3(self):  
9)         print("Sub Child Method")  
10) c=CC()  
11) c.m1()  
12) c.m2()  
13) c.m3()
```

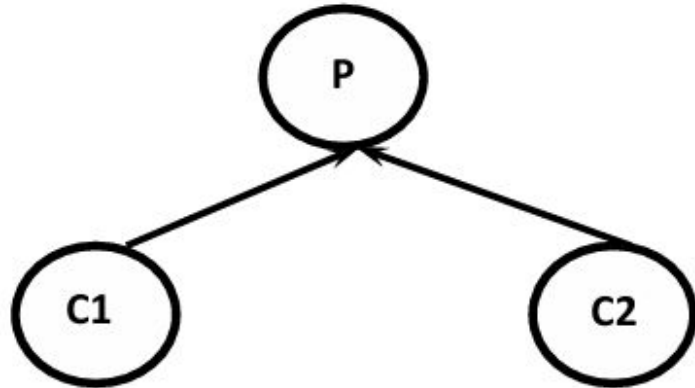


Multi – Level Inheritance

Object Oriented Programming(OOP) in Python

3. Hierarchical Inheritance:

The concept of inheriting properties from one class into multiple classes which are present at same level is known as Hierarchical Inheritance



**Hierarchical
Inheritance**

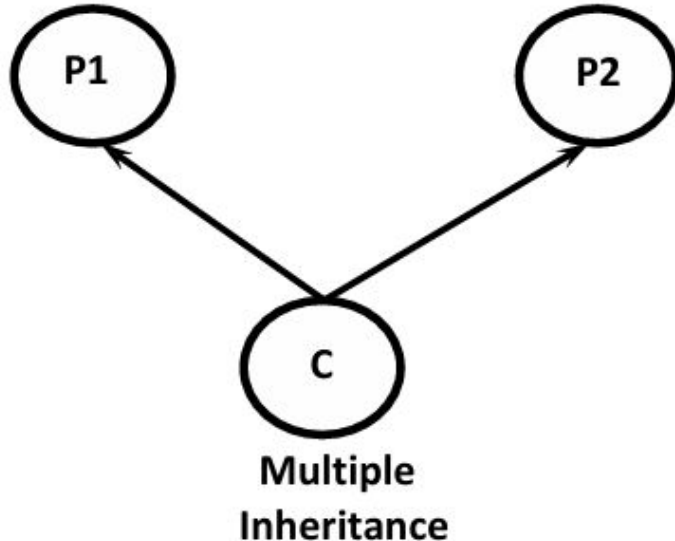
Object Oriented Programming(OOP) in Python

```
1) class P:
2)     def m1(self):
3)         print("Parent Method")
4) class C1(P):
5)     def m2(self):
6)         print("Child1 Method")
7) class C2(P):
8)     def m3(self):
9)         print("Child2 Method")
10) c1=C1()
11) c1.m1()
12) c1.m2()
13) c2=C2()
14) c2.m1()
15) c2.m3()
```

Object Oriented Programming(OOP) in Python

4. Multiple Inheritance:

The concept of inheriting the properties from multiple classes into a single class at a time, is known as multiple inheritance.



Object Oriented Programming(OOP) in Python

```
1) class P1:
2)     def m1(self):
3)         print("Parent1 Method")
4) class P2:
5)     def m2(self):
6)         print("Parent2 Method")
7) class C(P1,P2):
8)     def m3(self):
9)         print("Child2 Method")
10) c=C()
11) c.m1()
12) c.m2()
13) c.m3()
```

If the same method is inherited from both parent classes, then Python will always consider the order of Parent classes in the declaration of the child class.

class C(P1,P2): ==> P1 method will be considered

class C(P2,P1): ==> P2 method will be considered

Object Oriented Programming(OOP) in Python

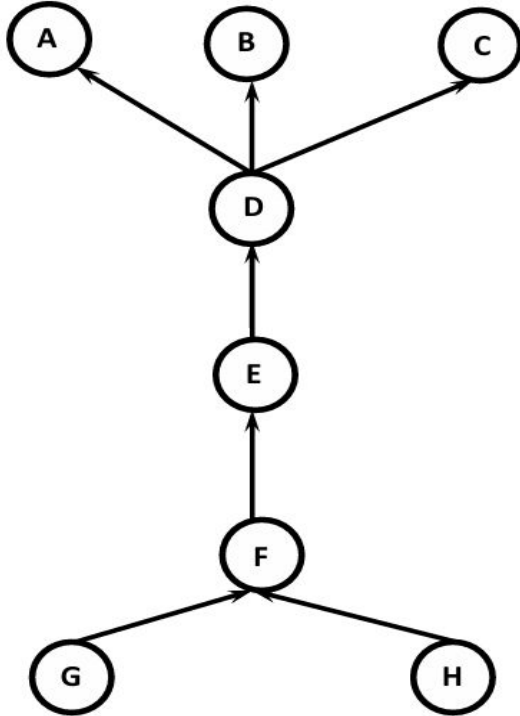
Eg:

```
1) class P1:  
2)     def m1(self):  
3)         print("Parent1 Method")  
4) class P2:  
5)     def m1(self):  
6)         print("Parent2 Method")  
7) class C(P1,P2):  
8)     def m2(self):  
9)         print("Child Method")  
10) c=C()  
11) c.m1()  
12) c.m2()
```

Object Oriented Programming(OOP) in Python

5. Hybrid Inheritance:

Combination of Single, Multi level, multiple and Hierarchical inheritance is known as Hybrid Inheritance.



Object Oriented Programming(OOP) in Python

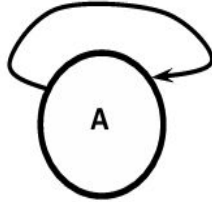
6. Cyclic Inheritance:

The concept of inheriting properties from one class to another class in cyclic way, is called Cyclic inheritance. Python won't support for Cyclic Inheritance of course it is really not required.

Eg - 1:

```
class A(A):pass
```

NameError: name 'A' is not defined



Eg - 2:

```
1) class A(B):  
2)     pass  
3) class B(A):  
4)     pass
```

NameError: name 'B' is not defined



Object Oriented Programming(OOP) in Python

super() Method:

super() is a built-in method which is useful to call the super class constructors, variables and methods from the child class.

Demo Program-1 for super():

```
1) class Person:
2)     def __init__(self,name,age):
3)         self.name=name
4)         self.age=age
5)     def display(self):
6)         print('Name:',self.name)
7)         print('Age:',self.age)
8)
9) class Student(Person):
10)    def __init__(self,name,age,rollno,marks):
11)        super().__init__(name,age)
12)        self.rollno=rollno
13)        self.marks=marks
14)
15)    def display(self):
16)        super().display()
17)        print('Roll No:',self.rollno)
18)        print('Marks:',self.marks)
19)
20) s1=Student('Durga',22,101,90)
```

```
21) s1.display()
```

Object Oriented Programming(OOP) in Python

Demo Program-2 for super():

```
1) class P:
2)     a=10
3)     def __init__(self):
4)         self.b=10
5)     def m1(self):
6)         print('Parent instance method')
7)     @classmethod
8)     def m2(cls):
9)         print('Parent class method')
10)    @staticmethod
11)    def m3():
12)        print('Parent static method')
13)
14) class C(P):
15)     a=888
16)     def __init__(self):
17)         self.b=999
18)         super().__init__()
19)         print(super().a)
20)         super().m1()
21)         super().m2()
22)         super().m3()
23)
24) c=C()
```

Object Oriented Programming(OOP) in Python

How to call method of a particular Super class:

We can use the following approaches

1. super(D,self).m1()

It will call m1() method of super class of D.

2. A.m1(self)

It will call A class m1() method

```
1) class A:
2)     def m1(self):
3)         print('A class Method')
4) class B(A):
5)     def m1(self):
6)         print('B class Method')
7) class C(B):
8)     def m1(self):
9)         print('C class Method')
10) class D(C):
11)     def m1(self):
12)         print('D class Method')
13) class E(D):
14)     def m1(self):
15)         A.m1(self)
16)
17) e=E()
18) e.m1()
```

Object Oriented Programming(OOP) in Python

Various Important Points about super():

Case-1: From child class we are not allowed to access parent class instance variables by using `super()`, Compulsory we should use `self` only.

But we can access parent class static variables by using `super()`.

Eg:

```
1) class P:
2)     a=10
3)     def __init__(self):
4)         self.b=20
5)
6) class C(P):
7)     def m1(self):
8)         print(super().a)#valid
9)         print(self.b)#valid
10)        print(super().b)#invalid
11) c=C()
12) c.m1()
```

Object Oriented Programming(OOP) in Python

Case-2: From child class constructor and instance method, we can access parent class instance method,static method and class method by using super()

```
1) class P:
2)     def __init__(self):
3)         print('Parent Constructor')
4)     def m1(self):
5)         print('Parent instance method')
6)     @classmethod
7)     def m2(cls):
8)         print('Parent class method')
9)     @staticmethod
10)    def m3():
11)        print('Parent static method')
12)
13) class C(P):
14)     def __init__(self):
15)         super().__init__()
16)         super().m1()
17)         super().m2()
18)         super().m3()
19)
20)     def m1(self):
21)         super().__init__()
22)         super().m1()
23)         super().m2()
24)         super().m3()
25)
26) c=C()
27) c.m1()
```

Object Oriented Programming(OOP) in Python

Case-3: From child class, class method we cannot access parent class instance methods and constructors by using `super()` directly(but indirectly possible). But we can access parent class static and class methods.

```
1) class P:
2)     def __init__(self):
3)         print('Parent Constructor')
4)     def m1(self):
5)         print('Parent instance method')
6)     @classmethod
7)     def m2(cls):
8)         print('Parent class method')
9)     @staticmethod
10)    def m3():
11)        print('Parent static method')
12)
13) class C(P):
14)     @classmethod
15)     def m1(cls):
16)         #super().__init__()--->invalid
17)         #super().m1()--->invalid
18)         super().m2()
19)         super().m3()
20)
21) C.m1()
```

Object Oriented Programming(OOP) in Python

From Class Method of Child class,how to call parent class instance methods and constructors:

```
1) class A:
2)     def __init__(self):
3)         print('Parent constructor')
4)
5)     def m1(self):
6)         print('Parent instance method')
7)
8) class B(A):
9)     @classmethod
10)    def m2(cls):
11)        super(B,cls).__init__(cls)
12)        super(B,cls).m1(cls)
13)
14) B.m2()
```

Object Oriented Programming(OOP) in Python

Case-4: In child class static method we are not allowed to use super() generally (But in special way we can use)

```
1) class P:
2)     def __init__(self):
3)         print('Parent Constructor')
4)     def m1(self):
5)         print('Parent instance method')
6)     @classmethod
7)     def m2(cls):
8)         print('Parent class method')
9)     @staticmethod
10)    def m3():
11)        print('Parent static method')
12)
13) class C(P):
14)     @staticmethod
15)     def m1():
16)         super().m1()-->invalid
17)         super().m2()--->invalid
18)         super().m3()--->invalid
19)
20) C.m1()
```

RuntimeError: super(): no arguments

Object Oriented Programming(OOP) in Python

How to call parent class static method from child class static method by using super():

```
1) class A:  
2)  
3)     @staticmethod  
4)     def m1():  
5)         print('Parent static method')  
6)  
7) class B(A):  
8)     @staticmethod  
9)     def m2():  
10)         super(B,B).m1()  
11)  
12) B.m2()
```

Output: Parent static method

Object Oriented Programming(OOP) in Python

Polymorphism

Poly means many. Morphs means forms.

Polymorphism means 'Many Forms'.

Eg1: Yourself is best example of polymorphism. In front of Your parents You will have one type of behaviour and with friends another type of behaviour. Same person but different behaviours at different places, which is nothing but polymorphism.

Eg2: + operator acts as concatenation and arithmetic addition

Eg3: * operator acts as multiplication and repetition operator

Eg4: The Same method with different implementations in Parent class and child classes. (overriding)

Object Oriented Programming(OOP) in Python

Related to polymorphism the following 4 topics are important

1. Duck Typing Philosophy of Python

2. Overloading

1. Operator Overloading
2. Method Overloading
3. Constructor Overloading

3. Overriding

1. Method overriding
2. constructor overriding

Object Oriented Programming(OOP) in Python

1. Duck Typing Philosophy of Python:

In Python we cannot specify the type explicitly. Based on provided value at runtime the type will be considered automatically. Hence Python is considered as Dynamically Typed Programming Language.

```
def f1(obj):  
    obj.talk()
```

What is the type of obj? We cannot decide at the beginning. At runtime we can pass any type. Then how we can decide the type?

At runtime if 'it walks like a duck and talks like a duck, it must be duck'. Python follows this principle. This is called Duck Typing Philosophy of Python.

Object Oriented Programming(OOP) in Python

Demo Program:

```
1) class Duck:
2)     def talk(self):
3)         print('Quack.. Quack..')
4)
5) class Dog:
6)     def talk(self):
7)         print('Bow Bow..')
8)
9) class Cat:
10)    def talk(self):
11)        print('Moew Moew ..')
12)
13) class Goat:
14)    def talk(self):
15)        print('Myaah Myaah ..')
16)
17) def f1(obj):
18)     obj.talk()
19)
20) l=[Duck(),Cat(),Dog(),Goat()]
21) for obj in l:
22)     f1(obj)
```

Object Oriented Programming(OOP) in Python

The problem in this approach is if obj does not contain talk() method then we will get **AttributeError**

Eg:

```
1) class Duck:
2)     def talk(self):
3)         print('Quack.. Quack..')
4)
5) class Dog:
6)     def bark(self):
7)         print('Bow Bow..')
8) def f1(obj):
9)     obj.talk()
10)
11) d=Duck()
12) f1(d)
13)
14) d=Dog()
15) f1(d)
```

Demo Program with hasattr() function:

```
1) class Duck:
2)     def talk(self):
3)         print('Quack.. Quack..')
4)
5) class Human:
6)     def talk(self):
7)         print('Hello Hi...')
8)
9) class Dog:
10)    def bark(self):
11)        print('Bow Bow..')
12)
13) def f1(obj):
14)     if hasattr(obj,'talk'):
15)         obj.talk()
16)     elif hasattr(obj,'bark'):
17)         obj.bark()
18)
19) d=Duck()
20) f1(d)
21)
22) h=Human()
23) f1(h)
24)
25) d=Dog()
26) f1(d)
27) Myaah Myaah Myaah...
```

Object Oriented Programming(OOP) in Python

Overloading:

We can use same operator or methods for different purposes.

Eg1: + operator can be used for Arithmetic addition and String concatenation

```
print(10+20)#30  
print('durga'+ 'soft')#durgasoft
```

Eg2: * operator can be used for multiplication and string repetition purposes.

```
print(10*20)#200  
print('durga'*3)#durgadurgadurga
```

Eg3: We can use deposit() method to deposit cash or cheque or dd

```
deposit(cash)  
deposit(cheque)  
deposit(dd)
```

There are 3 types of overloading

1. Operator Overloading
2. Method Overloading
3. Constructor Overloading

Object Oriented Programming(OOP) in Python

1. Operator Overloading:

We can use the same operator for multiple purposes, which is nothing but operator overloading.

Python supports operator overloading.

Eg1: + operator can be used for Arithmetic addition and String concatenation

```
print(10+20)#30  
print('durga'+ 'soft')#durgasoft
```

Eg2: * operator can be used for multiplication and string repetition purposes.

```
print(10*20)#200  
print('durga'*3)#durgadurgadurga
```


Object Oriented Programming(OOP) in Python

Demo program to use + operator for our class objects:

```
1) class Book:  
2)     def __init__(self,pages):  
3)         self.pages=pages  
4)  
5) b1=Book(100)  
6) b2=Book(200)  
7) print(b1+b2)
```

D:\durga_classes>py test.py

Traceback (most recent call last):

File "test.py", line 7, in <module>

print(b1+b2)

TypeError: unsupported operand type(s) for +: 'Book' and 'Book'

Object Oriented Programming(OOP) in Python

We can overload + operator to work with Book objects also. i.e Python supports Operator Overloading.

For every operator Magic Methods are available. To overload any operator we have to override that Method in our class.

Internally + operator is implemented by using `__add__()` method. This method is called magic method for + operator. We have to override this method in our class.

Object Oriented Programming(OOP) in Python

Demo program to overload + operator for our Book class objects:

```
1) class Book:
2)     def __init__(self,pages):
3)         self.pages=pages
4)
5)     def __add__(self,other):
6)         return self.pages+other.pages
7)
8) b1=Book(100)
9) b2=Book(200)
10) print('The Total Number of Pages:',b1+b2)
```

Output: The Total Number of Pages: 300

Object Oriented Programming(OOP) in Python

The following is the list of operators and corresponding magic methods.

```
+ ---> object.__add__(self,other)
- ---> object.__sub__(self,other)
* ---> object.__mul__(self,other)
/ ---> object.__div__(self,other)
// ---> object.__floordiv__(self,other)
% ---> object.__mod__(self,other)
** ---> object.__pow__(self,other)
+= ---> object.__iadd__(self,other)
-= ---> object.__isub__(self,other)
*= ---> object.__imul__(self,other)
/= ---> object.__idiv__(self,other)
//= ---> object.__ifloordiv__(self,other)
%= ---> object.__imod__(self,other)
**= ---> object.__ipow__(self,other)
< ---> object.__lt__(self,other)
<= ---> object.__le__(self,other)
> ---> object.__gt__(self,other)
>= ---> object.__ge__(self,other)
== ---> object.__eq__(self,other)
!= ---> object.__ne__(self,other)
```

Object Oriented Programming(OOP) in Python

Overloading > and <= operators for Student class objects:

```
1) class Student:
2)     def __init__(self,name,marks):
3)         self.name=name
4)         self.marks=marks
5)     def __gt__(self,other):
6)         return self.marks>other.marks
7)     def __le__(self,other):
8)         return self.marks<=other.marks
9)
10)
11) print("10>20 =",10>20)
12) s1=Student("Durga",100)
13) s2=Student("Ravi",200)
14) print("s1>s2=",s1>s2)
15) print("s1<s2=",s1<s2)
16) print("s1<=s2=",s1<=s2)
17) print("s1>=s2=",s1>=s2)
```

Output:

```
10>20 = False
s1>s2= False
s1<s2= True
s1<=s2= True
s1>=s2= False
```

Object Oriented Programming(OOP) in Python

Program to overload multiplication operator to work on Employee objects:

```
1) class Employee:
2)     def __init__(self,name,salary):
3)         self.name=name
4)         self.salary=salary
5)     def __mul__(self,other):
6)         return self.salary*other.days
7)
8) class TimeSheet:
9)     def __init__(self,name,days):
10)         self.name=name
11)         self.days=days
12)
13) e=Employee('Durga',500)
14) t=TimeSheet('Durga',25)
15) print('This Month Salary:',e*t)
```

Output: This Month Salary: 12500

Object Oriented Programming(OOP) in Python

2. Method Overloading:

If 2 methods having same name but different type of arguments then those methods are said to be overloaded methods.

Eg: m1(int a)
m1(double d)

But in Python Method overloading is not possible.

If we are trying to declare multiple methods with same name and different number of arguments then Python will always consider only last method.

Object Oriented Programming(OOP) in Python

Demo Program:

```
1) class Test:
2)     def m1(self):
3)         print('no-arg method')
4)     def m1(self,a):
5)         print('one-arg method')
6)     def m1(self,a,b):
7)         print('two-arg method')
8)
9) t=Test()
10) #t.m1()
11) #t.m1(10)
12) t.m1(10,20)
```

Output: two-arg method

In the above program python will consider only last method.

Object Oriented Programming(OOP) in Python

How we can handle overloaded method requirements in Python:

Most of the times, if method with variable number of arguments required then we can handle with default arguments or with variable number of argument methods.

Demo Program with Default Arguments:

```
1) class Test:
2)     def sum(self,a=None,b=None,c=None):
3)         if a!=None and b!= None and c!= None:
4)             print('The Sum of 3 Numbers:',a+b+c)
5)         elif a!=None and b!= None:
6)             print('The Sum of 2 Numbers:',a+b)
7)         else:
8)             print('Please provide 2 or 3 arguments')
9)
10) t=Test()
11) t.sum(10,20)
12) t.sum(10,20,30)
13) t.sum(10)
```

Object Oriented Programming(OOP) in Python

Demo Program with Variable Number of Arguments:

```
1) class Test:
2)     def sum(self,*a):
3)         total=0
4)         for x in a:
5)             total=total+x
6)         print('The Sum:',total)
7)
8)
9) t=Test()
10) t.sum(10,20)
11) t.sum(10,20,30)
12) t.sum(10)
13) t.sum()
```

Object Oriented Programming(OOP) in Python

3. Constructor Overloading:

Constructor overloading is not possible in Python.

If we define multiple constructors then the last constructor will be considered.

```
1) class Test:
2)     def __init__(self):
3)         print('No-Arg Constructor')
4)
5)     def __init__(self,a):
6)         print('One-Arg constructor')
7)
8)     def __init__(self,a,b):
9)         print('Two-Arg constructor')
10) #t1=Test()
11) #t1=Test(10)
12) t1=Test(10,20)
```

Output: Two-Arg constructor

In the above program only Two-Arg Constructor is available.

Object Oriented Programming(OOP) in Python

But based on our requirement we can declare constructor with default arguments and variable number of arguments.

Constructor with Default Arguments:

```
1) class Test:  
2)     def __init__(self,a=None,b=None,c=None):  
3)         print('Constructor with 0|1|2|3 number of arguments')  
4)  
5) t1=Test()  
6) t2=Test(10)  
7) t3=Test(10,20)  
8) t4=Test(10,20,30)
```

Output:

Constructor with 0|1|2|3 number of arguments
Constructor with 0|1|2|3 number of arguments
Constructor with 0|1|2|3 number of arguments
Constructor with 0|1|2|3 number of arguments

Object Oriented Programming(OOP) in Python

Constructor with Variable Number of Arguments:

```
1) class Test:  
2)     def __init__(self,*a):  
3)         print('Constructor with variable number of arguments')  
4)  
5) t1=Test()  
6) t2=Test(10)  
7) t3=Test(10,20)  
8) t4=Test(10,20,30)  
9) t5=Test(10,20,30,40,50,60)
```

Output:

Constructor with variable number of arguments
Constructor with variable number of arguments
Constructor with variable number of arguments
Constructor with variable number of arguments
Constructor with variable number of arguments

Object Oriented Programming(OOP) in Python

Method overriding:

What ever members available in the parent class are bydefault available to the child class through inheritance. If the child class not satisfied with parent class implementation then child class is allowed to redefine that method in the child class based on its requirement. This concept is called overriding.

Overriding concept applicable for both methods and constructors.

Demo Program for Method overriding:

```
1) class P:
2)     def property(self):
3)         print('Gold+Land+Cash+Power')
4)     def marry(self):
5)         print('Appalamma')
6) class C(P):
7)     def marry(self):
8)         print('Katrina Kaif')
9)
10) c=C()
11) c.property()
12) c.marry()
```

Output:

Gold+Land+Cash+Power
Katrina Kaif

Object Oriented Programming(OOP) in Python

From Overriding method of child class,we can call parent class method also by using `super()` method.

```
1) class P:
2)     def property(self):
3)         print('Gold+Land+Cash+Power')
4)     def marry(self):
5)         print('Appalamma')
6) class C(P):
7)     def marry(self):
8)         super().marry()
9)         print('Katrina Kaif')
10)
11) c=C()
12) c.property()
13) c.marry()
```

Output:

Gold+Land+Cash+Power

Appalamma

Katrina Kaif

Object Oriented Programming(OOP) in Python

Demo Program for Constructor overriding:

```
1) class P:  
2)     def __init__(self):  
3)         print('Parent Constructor')  
4)  
5) class C(P):  
6)     def __init__(self):  
7)         print('Child Constructor')  
8)  
9) c=C()
```

Output: Child Constructor

In the above example,if child class does not contain constructor then parent class constructor will be executed

Object Oriented Programming(OOP) in Python

From child class constructor we can call parent class constructor by using `super()` method.

Demo Program to call Parent class constructor by using `super()`:

```
1) class Person:
2)     def __init__(self,name,age):
3)         self.name=name
4)         self.age=age
5)
6) class Employee(Person):
7)     def __init__(self,name,age,eno,esal):
8)         super().__init__(name,age)
9)         self.eno=eno
10)        self.esal=esal
11)
12)    def display(self):
13)        print('Employee Name:',self.name)
14)        print('Employee Age:',self.age)
15)        print('Employee Number:',self.eno)
16)        print('Employee Salary:',self.esal)
17)
18) e1=Employee('Durga',48,872425,26000)
19) e1.display()
20) e2=Employee('Sunny',39,872426,36000)
21) e2.display()
```

Object Oriented Programming(OOP) in Python

Abstract Method:

Sometimes we don't know about implementation, still we can declare a method. Such type of methods are called abstract methods. i.e abstract method has only declaration but not implementation.

In python we can declare abstract method by using @abstractmethod decorator as follows.

```
@abstractmethod  
def m1(self): pass
```

@abstractmethod decorator present in abc module. Hence compulsory we should import abc module, otherwise we will get error.

abc==>abstract base class module

Eg:

```
1) from abc import *  
2) class Test:  
3)     @abstractmethod  
4)     def m1(self):  
5)         pass
```

Object Oriented Programming(OOP) in Python

Abstract class:

Some times implementation of a class is not complete, such type of partially implementation classes are called abstract classes. Every abstract class in Python should be derived from ABC class which is present in abc module.

Case-1:

```
1) from abc import *  
2) class Test:  
3)     pass  
4)  
5) t=Test()
```

In the above code we can create object for Test class b'z it is concrete class and it does not contain any abstract method.

Object Oriented Programming(OOP) in Python

Case-2:

```
1) from abc import *  
2) class Test(ABC):  
3)     pass  
4)  
5) t=Test()
```

In the above code we can create object,even it is derived from ABC class,b'z it does not contain any abstract method.

Object Oriented Programming(OOP) in Python

Case-3:

```
1) from abc import *
2) class Test(ABC):
3)     @abstractmethod
4)     def m1(self):
5)         pass
6)
7) t=Test()
```

TypeError: Can't instantiate abstract class Test with abstract methods m1

Object Oriented Programming(OOP) in Python

Case-4:

```
1) from abc import *
2) class Test:
3)     @abstractmethod
4)     def m1(self):
5)         pass
6)
7) t=Test()
```

We can create object even class contains abstract method b'z we are not extending ABC class.

Object Oriented Programming(OOP) in Python

Case-5:

```
1) from abc import *
2) class Test:
3)     @abstractmethod
4)     def m1(self):
5)         print('Hello')
6)
7) t=Test()
8) t.m1()
```

Output: Hello

Conclusion: If a class contains atleast one abstract method and if we are extending ABC class then instantiation is not possible.

Object Oriented Programming(OOP) in Python

"abstract class with abstract method instantiation is not possible"

Parent class abstract methods should be implemented in the child classes. otherwise we cannot instantiate child class.If we are not creating child class object then we won't get any error.

Case-1:

```
1) from abc import *
2) class Vehicle(ABC):
3)     @abstractmethod
4)     def noofwheels(self):
5)         pass
6)
7) class Bus(Vehicle): pass
```

It is valid b'z we are not creating Child class object

Object Oriented Programming(OOP) in Python

Case-2:

```
1) from abc import *
2) class Vehicle(ABC):
3)     @abstractmethod
4)     def noofwheels(self):
5)         pass
6)
7) class Bus(Vehicle): pass
8) b=Bus()
```

TypeError: Can't instantiate abstract class Bus with abstract methods noofwheels

Note: If we are extending abstract class and does not override its abstract method then child class is also abstract and instantiation is not possible.

Object Oriented Programming(OOP) in Python

Eg:

```
1) from abc import *
2) class Vehicle(ABC):
3)     @abstractmethod
4)     def noofwheels(self):
5)         pass
6)
7) class Bus(Vehicle):
8)     def noofwheels(self):
9)         return 7
10)
11) class Auto(Vehicle):
12)     def noofwheels(self):
13)         return 3
14) b=Bus()
15) print(b.noofwheels())#7
16)
17) a=Auto()
18) print(a.noofwheels())#3
```

Note: Abstract class can contain both abstract and non-abstract methods also.

Object Oriented Programming(OOP) in Python

Interfaces In Python:

In general if an abstract class contains only abstract methods such type of abstract class is considered as interface.

Demo program:

```
1) from abc import *
2) class DBInterface(ABC):
3)     @abstractmethod
4)     def connect(self):pass
5)
6)     @abstractmethod
7)     def disconnect(self):pass
```

Object Oriented Programming(OOP) in Python

Concrete class vs Abstract Class vs Interface:

1. If we don't know anything about implementation just we have requirement specification then we should go for interface.
2. If we are talking about implementation but not completely then we should go for abstract class.(partially implemented class)
3. If we are talking about implementation completely and ready to provide service then we should go for concrete class.

Object Oriented Programming(OOP) in Python

```
1) from abc import *
2) class CollegeAutomation(ABC):
3)     @abstractmethod
4)     def m1(self): pass
5)     @abstractmethod
6)     def m2(self): pass
7)     @abstractmethod
8)     def m3(self): pass
9) class AbsCls(CollegeAutomation):
10)     def m1(self):
11)         print('m1 method implementation')
12)     def m2(self):
13)         print('m2 method implementation')
14)
15) class ConcreteCls(AbsCls):
16)     def m3(self):
17)         print('m3 method implemnentation')
18)
19) c=ConcreteCls()
20) c.m1()
21) c.m2()
22) c.m3()
```

Object Oriented Programming(OOP) in Python

Public , Private and Protected attribute in python

```
class Person:  
    def __init__(self, name):  
        self.name = name # public attribute
```

```
class Person:  
    def __init__(self, name):  
        self._name = name # protected attribute
```

```
class Person:  
    def __init__(self, name):  
        self.__name = name # private attribute
```

Object Oriented Programming(OOP) in Python

`__str__()` method:

Whenever we are printing any object reference internally `__str__()` method will be called which is returns string in the following format

```
<__main__.classname object at 0x022144B0>
```

To return meaningful string representation we have to override `__str__()` method.

Demo Program:

```
1) class Student:
2)     def __init__(self,name,rollno):
3)         self.name=name
4)         self.rollno=rollno
5)
6)     def __str__(self):
7)         return 'This is Student with Name:{} and Rollno:{}'.format(self.name,self.rollno)
8)
9) s1=Student('Durga',101)
10) s2=Student('Ravi',102)
11) print(s1)
12) print(s2)
```